

Mount Kenya University



Unlocking Infinite Possibilities

NAME : STEPHEN KIMANI

ADMISSION: BSCCS/2024/57444

COURSE : BACHELOR OF SCIENCE IN COMPUTER SCIENCE

UNIT CODE: BIT 4138

UNIT NAME: ADVANCED CRYPTOGRAPHY

LECTURER: MICHAEL NYORO

SEMESTER /ACADEMIC YEAR: SEMESTER 6

**PROJECT DETAILS: IMPLEMENTATION AND ANALYSIS OF
CRYPTOGRAPHIC ALGORITHMS USING PYTHON**

Technologies Used:

- I. Python 3**
- II. Visual Studio Code**
- III. Git**
- IV. OpenSSL**
- V. PyCryptodome**
- VI. Windows PowerShell / Command Prompt**

ONLINE PORTFOLIO/ GITHUB LINK

<https://github.com/Kimani985/BIT4138-Advanced-Cryptography>

Contents

1	WEEK 1: Installation and Configuration of Cryptography Development Environment ...	8
1.1	Successful Installation of Python and Visual Studio Code	8
1.2	Installation of Cryptography Libraries Using pip	9
1.3	OpenSSL Version Verification	9
1.4	Python Test Script Execution	10
1.5	Python Test Script Execution	Error! Bookmark not defined.
1.6	Creative Task- Environment Checker	11
1.6.1	Reflection	11
2	WEEK 2: Implementation of Classical Encryption Algorithms	12
2.1	Caesar Cipher Implementation	12
2.2	Caesar Cipher Encryption and Decryption Results.....	13
2.3	User Input Validation	13
2.4	Vigenère Cipher Implementation	14
2.5	Cipher Testing Results	15
2.6	Creative Task - Cryptography Toolkit	15
2.6.1	Reflection	15
3	WEEK 3: Implementation of Stream Cipher Systems and Random Sequence Analysis	16
3.1	Linear Feedback Shift Register (LFSR) Implementation	16
3.2	Pseudorandom Sequence Generation	16
3.3	Statistical Randomness Testing.....	17
3.4	RC4-Style Stream Cipher Simulation	17
3.5	Encryption Performance Evaluation.....	17
3.6	Creative Task- Randomness Challenge Game	18
3.6.1	Reflection	18
4	WEEK 4: Advanced Encryption Standard (AES) and Block Cipher Operations	19
4.1	AES Encryption Script	19
4.2	AES Key Generation	20
4.3	File Encryption Demonstration	20

4.4	Successful File Decryption	20
4.5	AES Performance Evaluation	21
4.6	Creative Task - Secure File Locker.....	21
4.6.1	Reflection	21
5	WEEK 5: Implementation of RSA encryption and Key Management	22
	RSA Key Pair Generation.....	22
5.1	RSA Public Key Encryption	22
5.2	RSA Private Key Decryption	23
5.3	Secure Message Transmission	23
5.4	RSA Testing and Validation	23
5.5	Creative Task- Secure RSA Messenger	24
5.5.1	Reflection	24
6	WEEK 6 - SPN, S-Boxes and Non-Linearity.....	25
6.1	Class Activity 1: Manual SPN Simulation	25
6.2	Class Activity 2 – S-Box Analysis	25
6.3	Feistel vs SPN Comparison	26
6.4	Implementation of a Simple Substitution-Permutation Network (SPN)	26
6.5	Figure 34: Simple SPN cipher implementation and execution.	26
6.6	Creative Task 1 - SPN Visualizer	27
6.7	Creative Task 2 - Avalanche Effect Analyzer	27
6.7.1	Reflection	28
7	WEEK 7 - Cryptanalysis Techniques	29
7.1	Class Activity 1: Difference Analysis	29
7.2	Class Activity 2: Frequency Analysis	29
7.3	Class Activity 3: Probability Analysis	30
7.4	Class Activity 4: Differential Cryptanalysis Simulation	31
7.5	Creative Task – Cryptanalysis Toolkit.....	32
7.5.1	REFLECTION	32
8	WEEK 8: Public Key Infrastructure (PKI), Digital Certificates and Diffie–Hellman Key Exchange.....	33

8.1	Class Activity 1: PKI Component Identification	33
8.2	Class Activity 2 – Browser Certificate Analysis	34
8.3	OpenSSL Certificate Inspection	35
8.4	Creative Task 1 – Certificate Inspector	35
8.5	Practical Activity – Diffie–Hellman Key Exchange	36
8.6	Creative Task 2 – Secure Key Exchange Simulator	36
8.6.1	Reflection	37

LIST OF FIGURES

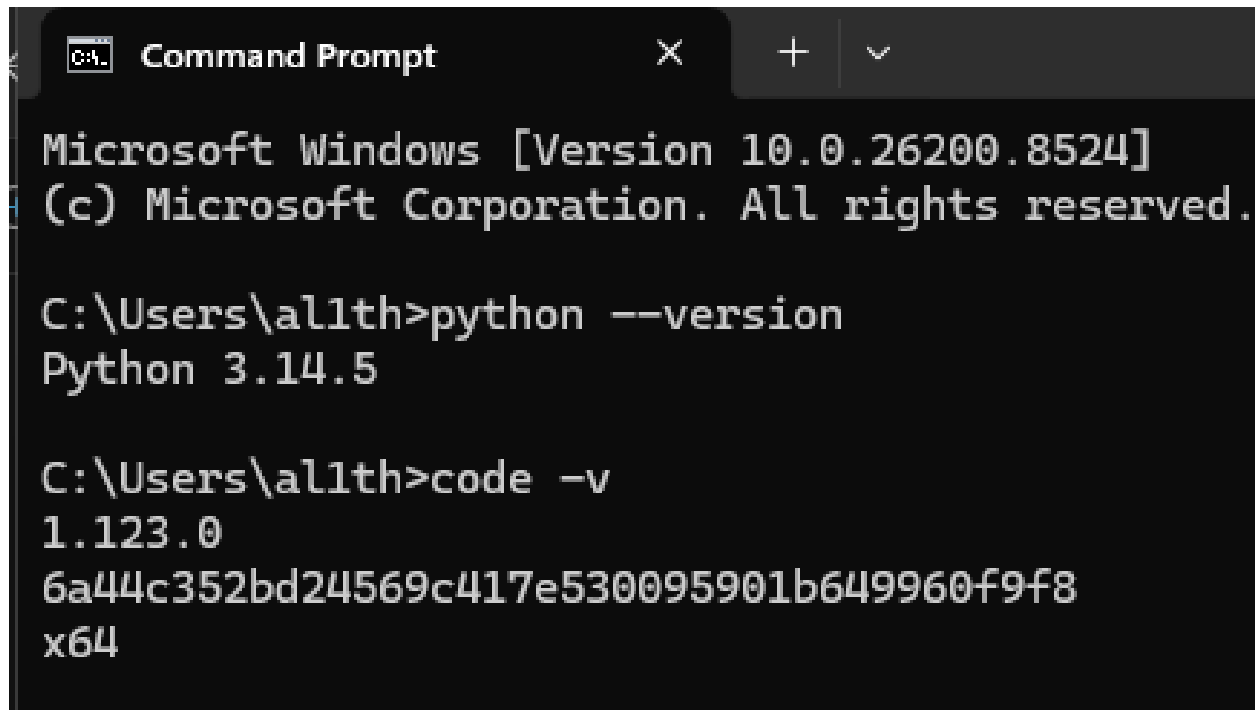
Figure 1: Successful installation of Python and Visual Studio Code for developing and testing cryptographic applications.	Error! Bookmark not defined.
Figure 2: Installation of cryptography libraries using pip, enabling implementation of encryption and decryption algorithms.	9
Figure 3: OpenSSL version verification confirming successful configuration of cryptographic command-line tools.....	9
Figure 4: Execution of a Python test script verifying the development environment is operational.	10
Figure 5: Initialization of GitHub repository for version control and project documentation.	10
Figure 6: Automated environment verification tool displaying the installed versions of Python, Git, and OpenSSL to confirm system readiness for cryptographic development.	Error! Bookmark not defined.
Figure 7: Caesar Cipher implementation demonstrating character shifting using a fixed encryption key.	12
Figure 8: Successful encryption and decryption outputs generated by the implemented cipher algorithm.	13
Figure 9: Input validation process preventing invalid plaintext entries during encryption. ...	13
Figure 10: Vigenère Cipher implementation demonstrating keyword-based encryption. ...	14
Figure 11: Cipher testing results obtained using different messages and encryption keys. ...	15
Figure 12: Interactive cryptography toolkit providing a menu-driven interface for selecting and executing classical encryption algorithms.	15
Figure 13: Implementation of a Linear Feedback Shift Register (LFSR) used to generate pseudorandom sequences.	16
Figure 14: Generated pseudorandom binary sequence produced by the implemented LFSR.	16
Figure 15: Statistical randomness testing results showing distribution of generated binary values.	17
Figure 16: RC4-style stream cipher simulation demonstrating encryption using a generated keystream.	17
Figure 17: Performance evaluation results showing execution time during encryption operations.	17
Figure 18: Interactive Randomness Challenge Game demonstrating binary sequence generation, user prediction, and statistical analysis of pseudorandom output.....	18
Figure 19: AES encryption program implementing symmetric key cryptography for secure file protection	19

Figure 20: AES symmetric key generation process used for encryption and decryption operations	20
Figure 21: Successful encryption of a text file using the generated AES key.	20
Figure 22: Successful decryption process restoring the original file contents.	20
Figure 23: AES performance evaluation showing execution time during encryption.	Error!
Bookmark not defined.	
Figure 24: Secure File Locker demonstrating user-driven AES encryption and successful recovery of encrypted file contents through decryption.	21
Figure 25: Generation of RSA public and private key pairs used for secure communication.	22
Figure 26: Encryption of plaintext using the RSA public key.	22
Figure 27: Successful recovery of the original message using the RSA private key.	Error!
Bookmark not defined.	
Figure 28: Simulation of secure communication using RSA public and private key cryptography.	Error! Bookmark not defined.
Figure 29: RSA testing and validation confirming successful encryption and decryption operations.	23
Figure 30: Secure RSA Messenger demonstrating encrypted communication between Alice and Bob through RSA public-key encryption and private-key decryption.	24
Figure 31: Manual SPN simulation showing substitution and permutation across two rounds.	25
Figure 32 : S-Box analysis showing mapping uniqueness and security evaluation.	25
Figure 33: Feistel and SPN structure comparison.	26
Figure 34: Simple SPN cipher implementation and execution.	26
Figure 35: SPN Visualizer illustrating the step-by-step transformation of plaintext through substitution and permutation stages to produce ciphertext.	27
Figure 36: Avalanche Effect Analyzer showing input difference analysis.	27
Figure 37: Difference analysis implementation and execution.	29
Figure 38 : Frequency analysis showing character occurrence counts.	29
Figure 39: Probability analysis showing likelihood calculation.	30
Figure 40: Differential cryptanalysis simulation showing plaintext differences.	31
Figure 41: Cryptanalysis Toolkit integrating difference, frequency, and probability analysis techniques.	32
Figure 42: PKI component identification displaying the major components and their roles in secure digital communication.	33
Figure 43: Browser certificate analysis showing the certificate issuer, subject, and validity period.	34
Figure 44: OpenSSL certificate inspection displaying certificate details and validity.	35

Figure 45: Certificate Inspector displaying certificate information in a structured format. .	35
Figure 46: Diffie–Hellman key exchange demonstrating successful generation of public keys and a shared secret.....	36
Figure 47:Secure Key Exchange Simulator demonstrating interactive Diffie–Hellman key exchange and shared secret verification.	36

1 WEEK 1: Installation and Configuration of Cryptography Development Environment

1.1 Successful Installation of Python and Visual Studio Code

A screenshot of a Windows Command Prompt window. The title bar reads "Command Prompt" with standard window controls. The text inside the window shows the system version and user information, followed by two commands and their outputs. The first command is 'python --version' which outputs 'Python 3.14.5'. The second command is 'code -v' which outputs '1.123.0' followed by a long alphanumeric hash and 'x64'.

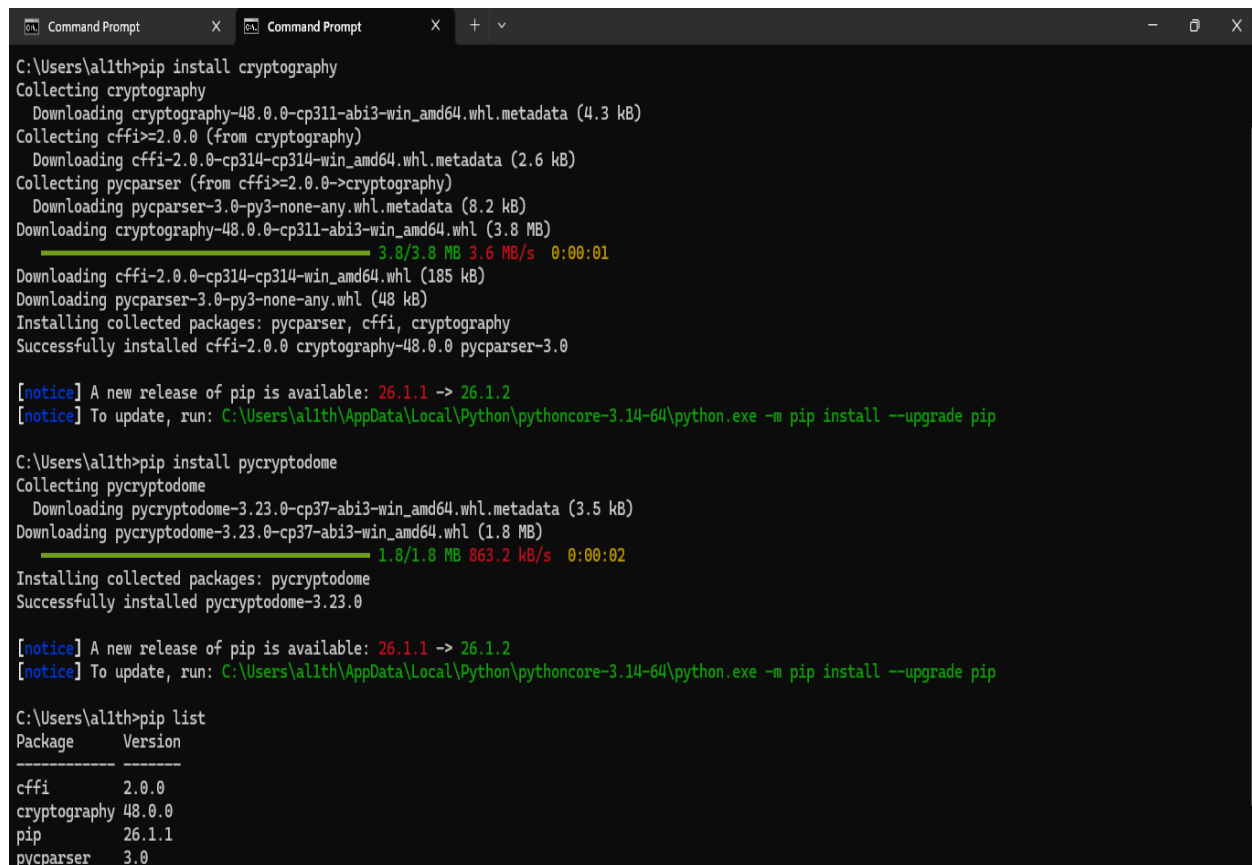
```
Microsoft Windows [Version 10.0.26200.8524]
(c) Microsoft Corporation. All rights reserved.

C:\Users\allth>python --version
Python 3.14.5

C:\Users\allth>code -v
1.123.0
6a44c352bd24569c417e530095901b649960f9f8
x64
```

Figure 1: Successful installation of Python and Visual Studio Code for developing and testing cryptographic applications.

1.2 Installation of Cryptography Libraries Using pip



```
C:\Users\al1th>pip install cryptography
Collecting cryptography
  Downloading cryptography-48.0.0-cp311-abi3-win_amd64.whl.metadata (4.3 kB)
Collecting cffi>=2.0.0 (from cryptography)
  Downloading cffi-2.0.0-cp314-cp314-win_amd64.whl.metadata (2.6 kB)
Collecting pycparser (from cffi>=2.0.0->cryptography)
  Downloading pycparser-3.0-py3-none-any.whl.metadata (8.2 kB)
Download cryptography-48.0.0-cp311-abi3-win_amd64.whl (3.8 MB)
  3.8/3.8 MB 3.6 MB/s 0:00:01
Download cffi-2.0.0-cp314-cp314-win_amd64.whl (185 kB)
Download pycparser-3.0-py3-none-any.whl (48 kB)
Installing collected packages: pycparser, cffi, cryptography
Successfully installed cffi-2.0.0 cryptography-48.0.0 pycparser-3.0

[notice] A new release of pip is available: 26.1.1 -> 26.1.2
[notice] To update, run: C:\Users\al1th\AppData\Local\Python\pythoncore-3.14-64\python.exe -m pip install --upgrade pip

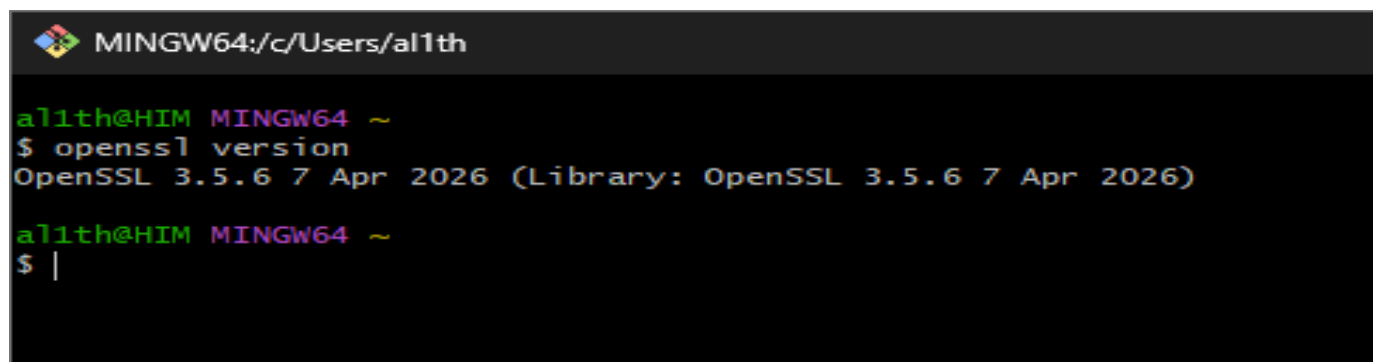
C:\Users\al1th>pip install pycryptodome
Collecting pycryptodome
  Downloading pycryptodome-3.23.0-cp37-abi3-win_amd64.whl.metadata (3.5 kB)
Download pycryptodome-3.23.0-cp37-abi3-win_amd64.whl (1.8 MB)
  1.8/1.8 MB 863.2 kB/s 0:00:02
Installing collected packages: pycryptodome
Successfully installed pycryptodome-3.23.0

[notice] A new release of pip is available: 26.1.1 -> 26.1.2
[notice] To update, run: C:\Users\al1th\AppData\Local\Python\pythoncore-3.14-64\python.exe -m pip install --upgrade pip

C:\Users\al1th>pip list
Package      Version
-----
cffi         2.0.0
cryptography 48.0.0
pip          26.1.1
pycparser    3.0
```

Figure 2: Installation of cryptography libraries using pip, enabling implementation of encryption and decryption algorithms.

1.3 OpenSSL Version Verification



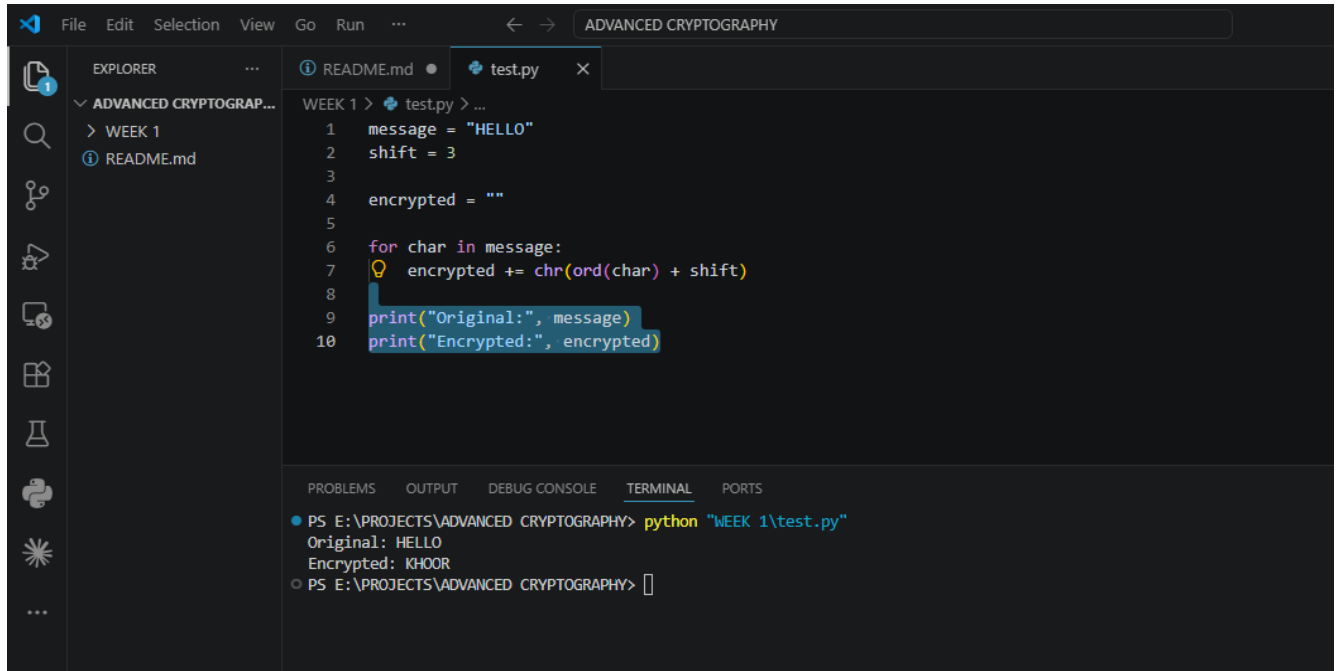
```
MINGW64:/c/Users/al1th

al1th@HIM MINGW64 ~
$ openssl version
OpenSSL 3.5.6 7 Apr 2026 (Library: OpenSSL 3.5.6 7 Apr 2026)

al1th@HIM MINGW64 ~
$ |
```

Figure 3: OpenSSL version verification confirming successful configuration of cryptographic command-line tools.

1.4 Python Test Script Execution



The screenshot shows the Visual Studio Code interface with a project named 'ADVANCED CRYPTOGRAPHY'. The Explorer sidebar on the left shows the project structure with 'WEEK 1' and 'README.md'. The main editor displays the file 'test.py' with the following Python code:

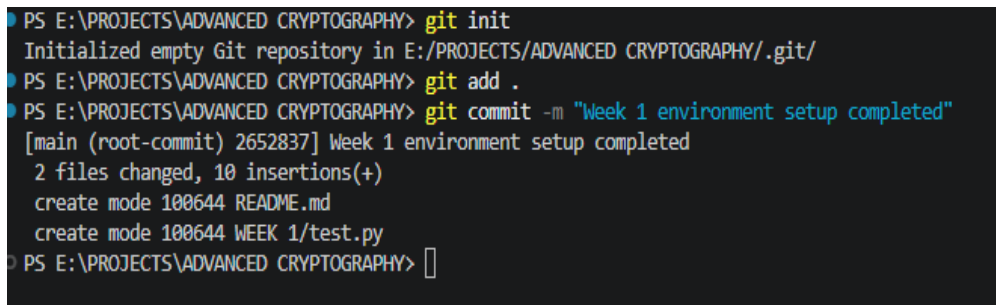
```
1 message = "HELLO"
2 shift = 3
3
4 encrypted = ""
5
6 for char in message:
7     encrypted += chr(ord(char) + shift)
8
9 print("Original:", message)
10 print("Encrypted:", encrypted)
```

The bottom panel shows the 'TERMINAL' tab with the following output:

```
PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY> python "WEEK 1\test.py"
Original: HELLO
Encrypted: KHOOR
PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY>
```

Figure 4: Execution of a Python test script verifying the development environment is operational.

1.5 Git Repository Initialization

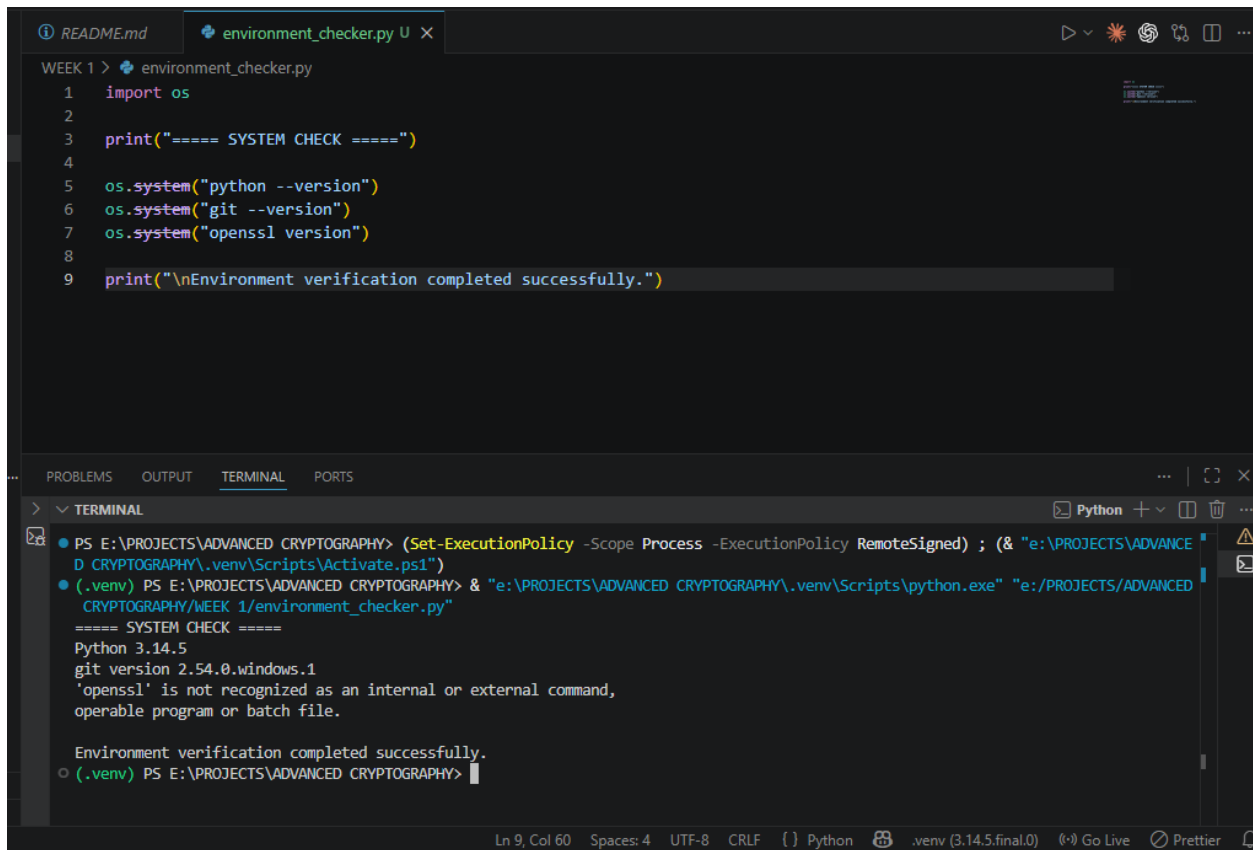


The screenshot shows a terminal window with the following commands and output:

```
PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY> git init
Initialized empty Git repository in E:/PROJECTS/ADVANCED CRYPTOGRAPHY/.git/
PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY> git add .
PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY> git commit -m "Week 1 environment setup completed"
[main (root-commit) 2652837] Week 1 environment setup completed
2 files changed, 10 insertions(+)
create mode 100644 README.md
create mode 100644 WEEK 1/test.py
PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY>
```

Figure 5: Initialization of GitHub repository for version control and project documentation.

1.6 Creative Task- Environment Checker



```
WEEK 1 > environment_checker.py
1 import os
2
3 print("==== SYSTEM CHECK ====")
4
5 os.system("python --version")
6 os.system("git --version")
7 os.system("openssl version")
8
9 print("\nEnvironment verification completed successfully.")
```

```
PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& "e:\PROJECTS\ADVANCED CRYPTOGRAPHY\Scripts\Activate.ps1")
(.venv) PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY> & "e:\PROJECTS\ADVANCED CRYPTOGRAPHY\Scripts\python.exe" "e:/PROJECTS/ADVANCED CRYPTOGRAPHY/WEEK 1/environment_checker.py"
==== SYSTEM CHECK ====
Python 3.14.5
git version 2.54.0.windows.1
'openssl' is not recognized as an internal or external command,
operable program or batch file.

Environment verification completed successfully.
(.venv) PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY>
```

Figure 6: Automated environment verification tool displaying the installed versions of Python, Git, and OpenSSL to confirm system readiness for cryptographic development.

1.6.1 Reflection

This week focused on setting up the cryptography development environment. Python, VS Code, OpenSSL, and required libraries were successfully installed and tested. GitHub was configured for version control. The environment is now ready for implementing and testing cryptographic algorithms.

2 WEEK 2: Implementation of Classical Encryption Algorithms

2.1 Caesar Cipher Implementation

```
WEEK 2 > 📄 classical_cipher.py > ...
1  def caesar_encrypt(text, shift):
2      result = ""
3
4      for char in text:
5          if char.isalpha():
6              result += chr((ord(char.upper()) - 65 + shift) % 26 + 65)
7          else:
8              result += char
9
10     return result
11
12
13     def caesar_decrypt(text, shift):
14         return caesar_encrypt(text, -shift)
15
16
17     message = input("Enter message: ")
18
19     if not message.replace(" ", "").isalpha():
20         print("Invalid input! Use letters only.")
21     else:
22         shift = int(input("Enter shift value: "))
23
24         encrypted = caesar_encrypt(message, shift)
25         decrypted = caesar_decrypt(encrypted, shift)
26
27         print("\nOriginal:", message)
28         print("Encrypted:", encrypted)
29         print("Decrypted:", decrypted)
```

Figure 7: Caesar Cipher implementation demonstrating character shifting using a fixed encryption key.

2.2 Caesar Cipher Encryption and Decryption Results

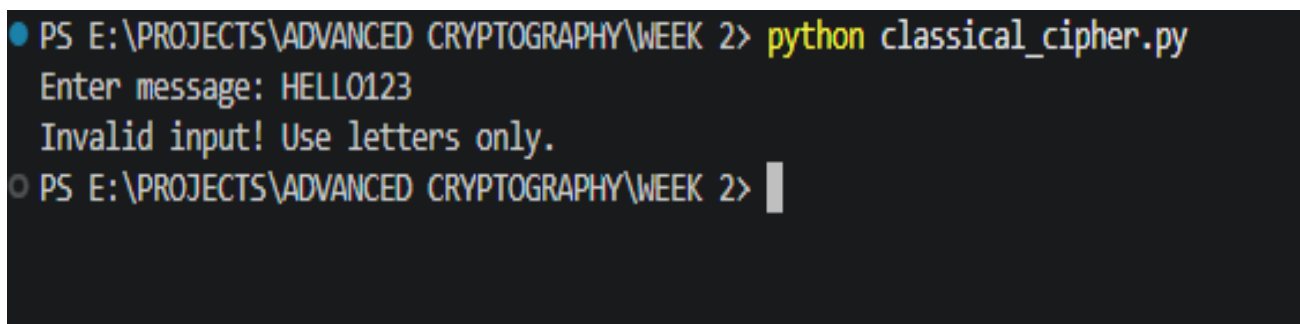


```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY> cd "WEEK 2"
PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY\WEEK 2> python classical_cipher.py
Enter message: HELLO
Enter shift value: 3

Original: HELLO
Encrypted: KHOOR
Decrypted: HELLO
PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY\WEEK 2> |
```

Figure 8: Successful encryption and decryption outputs generated by the implemented cipher algorithm.

2.3 User Input Validation



```
PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY\WEEK 2> python classical_cipher.py
Enter message: HELLO123
Invalid input! Use letters only.
PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY\WEEK 2> |
```

Figure 9: Input validation process preventing invalid plaintext entries during encryption.

2.4 Vigenère Cipher Implementation

```
WEEK 2 > vigenere_cipher.py > ...
1  def caesar_encrypt(text, shift):
2      result = ""
3
4      for char in text:
5          if char.isalpha():
6              result += chr((ord(char.upper()) - 65 + shift) % 26 + 65)
7          else:
8              result += char
9
10     return result
11
12
13  def vigenere_encrypt(text, key):
14      result = ""
15      key = key.upper()
16
17      key_index = 0
18
19      for char in text.upper():
20          if char.isalpha():
21              shift = ord(key[key_index % len(key)]) - 65
22              result += chr((ord(char) - 65 + shift) % 26 + 65)
23              key_index += 1
24          else:
25              result += char
26
27      return result
28
29
30  message = input("Enter message: ")
31
32  if not message.replace(" ", "").isalpha():
```

Figure 10: Vigenère Cipher implementation demonstrating keyword-based encryption.

2.5 Cipher Testing Results

```
● PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY> python "WEEK 2\vigenere_cipher.py"
Enter message: SECURITY

Caesar Cipher: VHFXULWB
Enter Vigenere key: LAMP
Vigenere Cipher: DEOJCIFN
○ PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY>
```

Figure 11: Cipher testing results obtained using different messages and encryption keys.

2.6 Creative Task - Cryptography Toolkit

```
● (.venv) PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY> python crypto_toolkit.py
E:\PROJECTS\ADVANCED CRYPTOGRAPHY\.venv\Scripts\python.exe: can't open file 'E:\PROJECTS\ADVANCED CRYPTOGRAPHY\crypto_toolkit.py':
[Errno 2] No such file or directory
(.venv) PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY> cd "E:\PROJECTS\ADVANCED CRYPTOGRAPHY\WEEK 2"
● >> python crypto_toolkit.py

===== CRYPTOGRAPHY TOOLKIT =====
1. Caesar Encrypt
2. Caesar Decrypt
3. Vigenère Encrypt
4. Exit
Enter your choice: 1
Enter message: HELLO
Enter shift value: 3

Encrypted Message: KHOOR

===== CRYPTOGRAPHY TOOLKIT =====
1. Caesar Encrypt
2. Caesar Decrypt
3. Vigenère Encrypt
4. Exit
Enter your choice: 4

Thank you for using the Cryptography Toolkit.
○ (.venv) PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY\WEEK 2>
```

Figure 12: Interactive cryptography toolkit providing a menu-driven interface for selecting and executing classical encryption algorithms.

2.6.1 Reflection

The implementation of Caesar and Vigenère ciphers demonstrated the importance of encryption keys in protecting information. Testing showed that Vigenère Cipher provides stronger security through keyword-based encryption, while Caesar Cipher remains vulnerable to simple cryptanalysis techniques.

3 WEEK 3: Implementation of Stream Cipher Systems and Random Sequence Analysis

3.1 Linear Feedback Shift Register (LFSR) Implementation

```
PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY\WEEK 3> python randomness.py
===== LFSR SIMULATION =====
Generated Sequence:
```

Figure 13: Implementation of a Linear Feedback Shift Register (LFSR) used to generate pseudorandom sequences.

3.2 Pseudorandom Sequence Generation

```
Generated Sequence:  
[1, 1, 0, 1, 0, 0, 1, 1, 1, 0, 1, 0, 0, 1, 1, 1, 0, 1, 0, 0, 1, 1, 1, 0, 1, 0, 0, 1, 1, 1, 0, 1, 0, 0, 1, 1, 1, 0, 1, 0, 0, 1, 1, 1, 0, 1, 0, 0,  
1, 1, 1, 0, 1, 0, 0, 1, 1, 1, 0, 1, 0, 0, 1, 1, 1, 0, 1, 0, 0, 1, 1, 1, 0, 1, 0, 0, 1, 1, 1, 0, 1, 0, 0, 1, 1, 1]
```

Figure 14: Generated pseudorandom binary sequence produced by the implemented LFSR.

3.3 Statistical Randomness Testing

```
===== RANDOMNESS TEST =====  
Number of Ones : 58  
Number of Zeros: 42  
Percentage of Ones : 58.00%  
Percentage of Zeros: 42.00%  
Result: Weak Randomness
```

Figure 15: Statistical randomness testing results showing distribution of generated binary values.

3.4 RC4-Style Stream Cipher Simulation

```
===== RC4 STYLE SIMULATION =====  
Message: HELLO  
Key Stream: [153, 253, 3, 156, 58]  
Encrypted: [209, 184, 79, 208, 117]
```

Figure 16: RC4-style stream cipher simulation demonstrating encryption using a generated keystream.

3.5 Encryption Performance Evaluation

```
===== PERFORMANCE RESULTS =====  
Execution Time: 7.724761962890625e-05 seconds  
D (.venv) PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY\WEEK 3>
```

Figure 17: Performance evaluation results showing execution time during encryption operations.

3.6 Creative Task- Randomness Challenge Game

```
(.venv) PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY\WEEK 2> cd "E:\PROJECTS\ADVANCED CRYPTOGRAPHY\WEEK 3"
(.venv) PS E:\PROpython randomness_game.pyHY\WEEK 3>
===== RANDOMNESS CHALLENGE GAME =====
Enter sequence length: 20

Sequence Generated Successfully!
Guess how many 1's are in the sequence: 10

===== RESULTS =====
Generated Sequence:
[1, 1, 0, 1, 0, 1, 1, 1, 0, 1, 1, 1, 0, 1, 1, 0, 0, 1, 0, 0]

Number of Ones : 12
Number of Zeros: 8

❌ Wrong guess.
Difference: 2

===== RANDOMNESS ANALYSIS =====
Percentage of Ones : 60.00%
Percentage of Zeros: 40.00%

Program Completed Successfully.
```

Figure 18: Interactive Randomness Challenge Game demonstrating binary sequence generation, user prediction, and statistical analysis of pseudorandom output.

3.6.1 Reflection

This week focused on pseudorandom sequence generation and stream cipher concepts. An LFSR was implemented to generate binary sequences, and statistical testing was performed to evaluate randomness. An RC4-style simulation demonstrated how stream ciphers use keystreams to encrypt data efficiently.

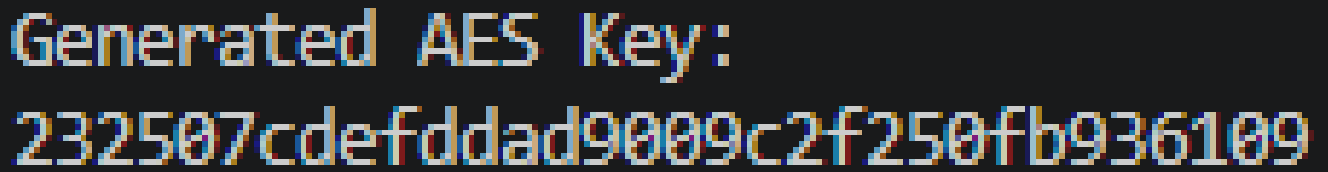
4 WEEK 4: Advanced Encryption Standard (AES) and Block Cipher Operations

4.1 AES Encryption Script

```
WEEK 4 > aes_encryption.py > ...
1  from Crypto.Cipher import AES
2  from Crypto.Random import get_random_bytes
3  import time
4
5  print("==== AES ENCRYPTION SYSTEM ====")
6
7  # Generate AES key
8  key = get_random_bytes(16)
9
10 print("\nGenerated AES Key:")
11 print(key.hex())
12
13 # Read file
14 with open("secret.txt", "rb") as file:
15     data = file.read()
16
17 # Padding
18 while len(data) % 16 != 0:
19     data += b' '
20
21 cipher = AES.new(key, AES.MODE_ECB)
22
23 start = time.time()
24
25 encrypted_data = cipher.encrypt(data)
26
27 with open("encrypted.bin", "wb") as file:
28     file.write(encrypted_data)
29
30 end = time.time()
31
32 print("\nFile Encrypted Successfully")
```

Figure 19: AES encryption program implementing symmetric key cryptography for secure file protection

4.2 AES Key Generation



Generated AES Key:
232507cdefddad9009c2f250fb936109

Figure 20: AES symmetric key generation process used for encryption and decryption operations

4.3 File Encryption Demonstration

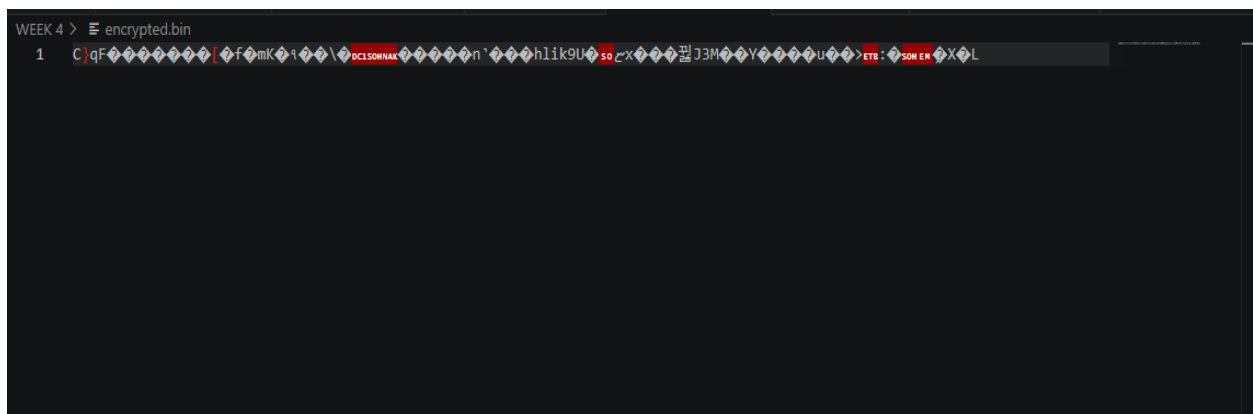
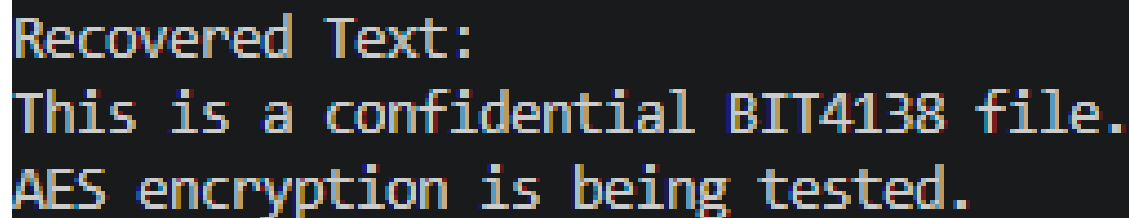


Figure 21: Successful encryption of a text file using the generated AES key.

4.4 Successful File Decryption



Recovered Text:
This is a confidential BIT4138 file.
AES encryption is being tested.

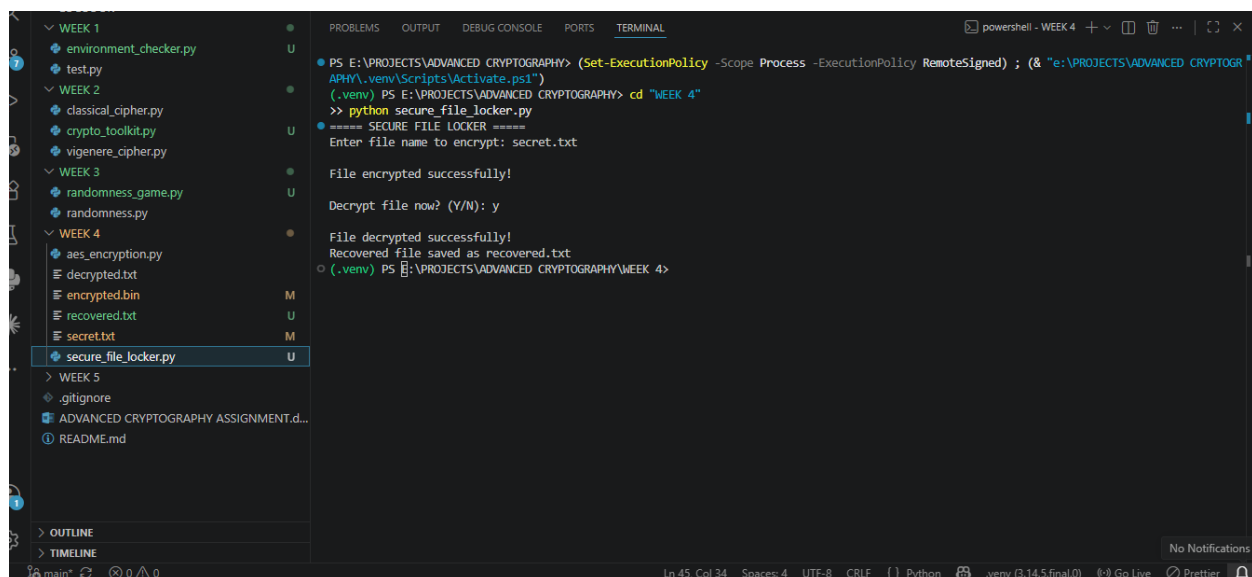
Figure 22: Successful decryption process restoring the original file contents.

4.5 AES Performance Evaluation

```
===== PERFORMANCE RESULTS =====
Encryption Time: 0.00034737586975097656 seconds
(.venv) PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY\WEEK 4>
```

Figure 23: AES performance evaluation showing execution time during encryption.

4.6 Creative Task - Secure File Locker



```
PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& "e:\PROJECTS\ADVANCED CRYPTOGRAPHY\WEEK 4\secure_file_locker.py")
(.venv) PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY> cd "WEEK 4"
>> python secure_file_locker.py
===== SECURE FILE LOCKER =====
Enter file name to encrypt: secret.txt

File encrypted successfully!

Decrypt file now? (Y/N): y

File decrypted successfully!
Recovered file saved as recovered.txt
(.venv) PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY\WEEK 4>
```

Figure 24: Secure File Locker demonstrating user-driven AES encryption and successful recovery of encrypted file contents through decryption.

4.6.1 Reflection

AES encryption was implemented to protect files using symmetric cryptography. Secure key generation, encryption, and decryption processes were successfully tested. The practical activity demonstrated the importance of key management and the effectiveness of AES in securing sensitive information.

5 WEEK 5: Implementation of RSA Encryption and Key Management

RSA Key Pair Generation

```
PTOGRAPHY\.venv\Scripts\Activate.ps1")
• (.venv) PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY\WEEK 5> python rsa_encryption.py
• (.venv) PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY\WEEK 5> python rsa_encryption.py
===== RSA ENCRYPTION SYSTEM =====

===== KEY GENERATION =====
Public Key Generated
Private Key Generated
Original Message:
```

Figure 25: Generation of RSA public and private key pairs used for secure communication.

5.1 RSA Public Key Encryption

```
===== PUBLIC KEY ENCRYPTION =====
Encrypted Message:
490faeb3e02d3bf10169d815bad4c25b32f2cb4cadcc2a0170d2553eb032757f76f750955230f487623fcec2f55f5c859be2be4dad41c432d89fcfd81ec890f439a287324773
c1c6cca3dfc0873e9de2cd24084e65df00fbd3b4b6bd469ca1e0f863043ff70e22a884a19fdb5b7a947e3556413fb0018db53d3dbdcf713ce443829c121b967c42a6848214bc7
495ae97d3909b841e41a33440c8e3622c70bcccc7a507c8fa0031c15cfc7352c01608a580a0b1b9142ac06eb23f559231773e8e16f6e94841d01a9bc10b1b9aba679d40656e6
decdfca9c89dd3485bf766187cb407443bb60fd7b96ef178877f547c49d6cb9384e5fe969fdb6ab047ec7
```

Figure 26: Encryption of plaintext using the RSA public key.

5.2 RSA Private Key Decryption

```
===== PRIVATE KEY DECRYPTION =====  
Recovered Message:  
HELLO BIT4138
```

Figure 27: Successful recovery of the original message using the RSA private key.

5.3 Secure Message Transmission

```
===== SECURE MESSAGE TRANSMISSION =====  
Alice encrypts with Bob's public key  
Bob decrypts with his private key
```

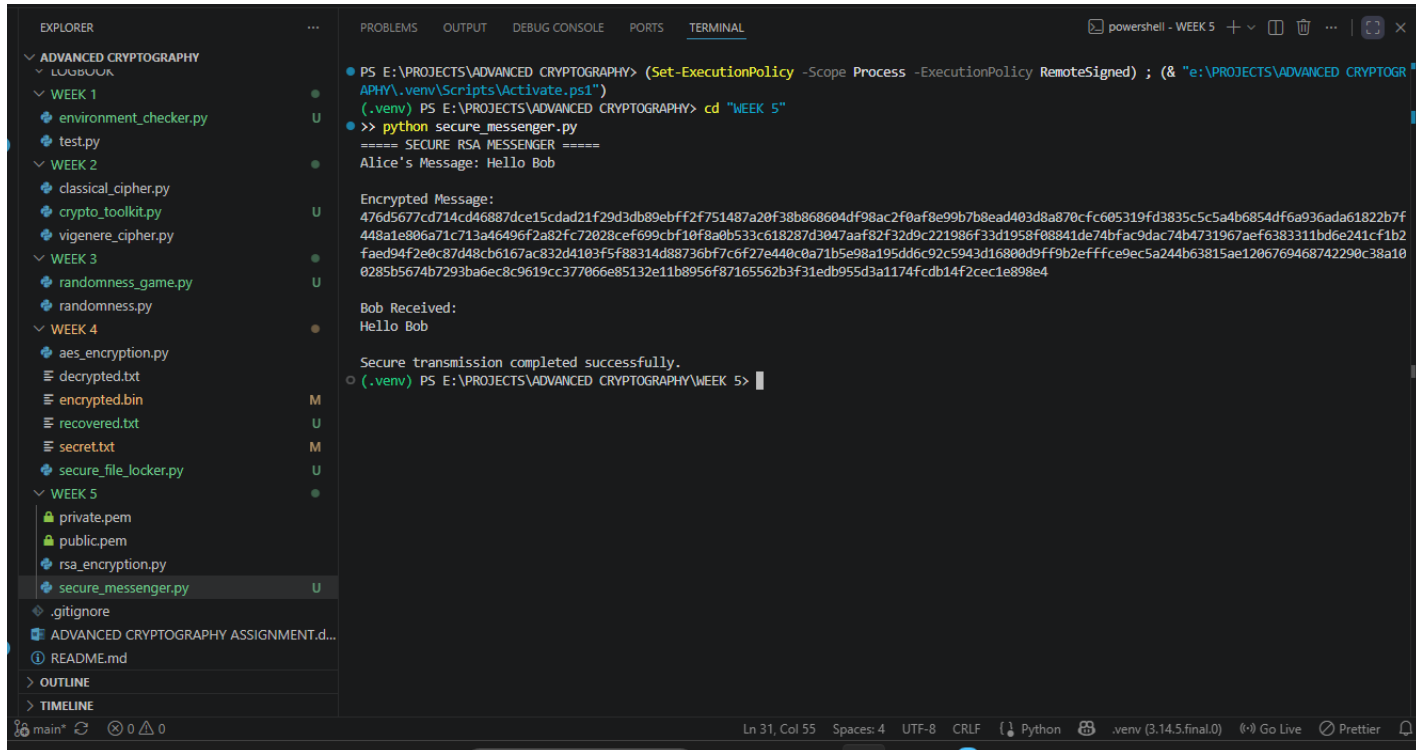
Figure 28: Simulation of secure communication using RSA public and private key cryptography.

5.4 RSA Testing and Validation

```
===== RSA VALIDATION =====  
Validation Successful  
(.venv) PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY\WEEK 5>
```

Figure 29: RSA testing and validation confirming successful encryption and decryption operations.

5.5 Creative Task- Secure RSA Messenger



```
PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& "e:\PROJECTS\ADVANCED CRYPTOGRAPHY\APHY\venv\Scripts\Activate.ps1")
(.venv) PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY> cd "WEEK 5"
>> python secure_messenger.py
===== SECURE RSA MESSENGER =====
Alice's Message: Hello Bob

Encrypted Message:
476d5677cd714cd46887dce15cdad21f29d3db89ebff2f751487a20f38b868604df98ac2f0af8e99b7b8ead403d8a870cfc605319fd3835c5c5a4b6854dff6a936ada61822b7f
448a1e806a71c713a46496f2a82fc72028cef699cbf18f8a0b533c618287d3047aaf82f32d9c221986f33d1958f08841de74bfac9dac74b4731967aef6383311bd6e241cf1b2
faed94f2e0c87d48cb6167ac832d4103f5f88314d88736bf7c6f27e440c0a71b5e98a195dd6c92c5943d16800d9ff9b2efffce9ec5a244b63815ae1206769468742290c38a10
0285b5674b7293ba6ec8c9619cc377066e85132e11b8956f87165562b3f31edb955d3a1174fcd14f2cec1e898e4

Bob Received:
Hello Bob

Secure transmission completed successfully.
(.venv) PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY\WEEK 5>
```

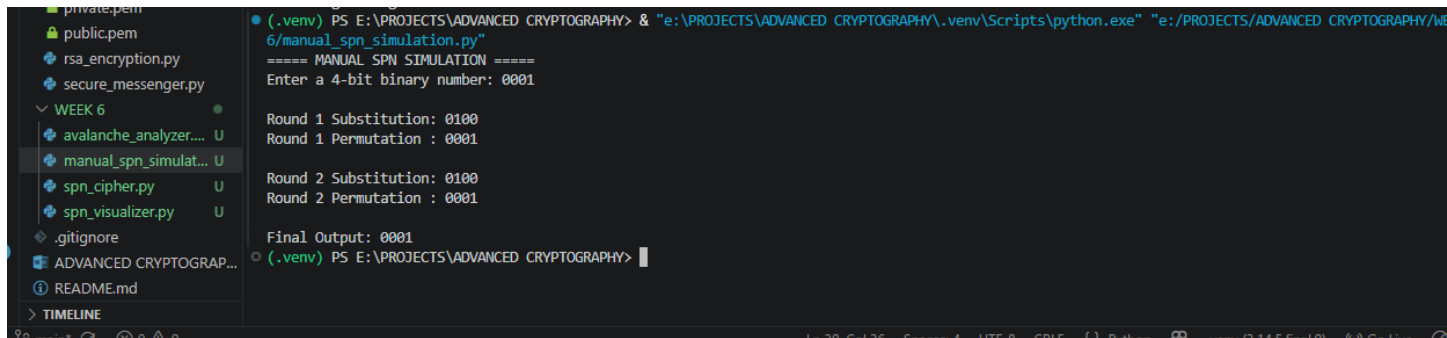
Figure 30: Secure RSA Messenger demonstrating encrypted communication between Alice and Bob through RSA public-key encryption and private-key decryption.

5.5.1 Reflection

RSA encryption demonstrated the use of public and private keys for secure communication. The Secure RSA Messenger successfully encrypted and decrypted messages, highlighting the practical application of asymmetric cryptography.

6 WEEK 6 - SPN, S-Boxes and Non-Linearity

6.1 Class Activity 1: Manual SPN Simulation



```
(.venv) PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY> & "e:\PROJECTS\ADVANCED CRYPTOGRAPHY\.venv\Scripts\python.exe" "e:/PROJECTS/ADVANCED CRYPTOGRAPHY/WEEK 6/manual_spn_simulation.py"
===== MANUAL SPN SIMULATION =====
Enter a 4-bit binary number: 0001

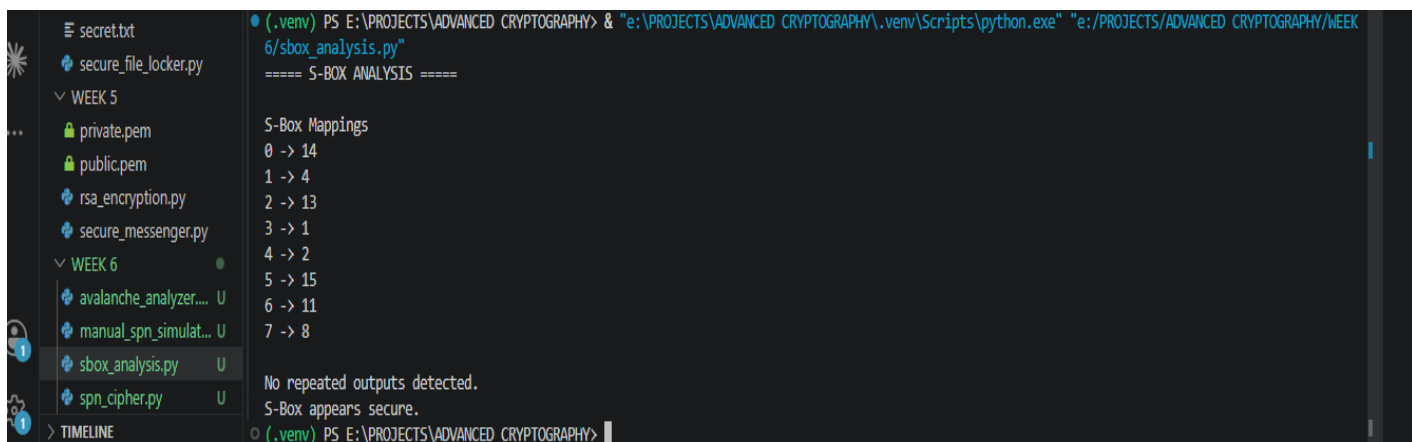
Round 1 Substitution: 0100
Round 1 Permutation : 0001

Round 2 Substitution: 0100
Round 2 Permutation : 0001

Final Output: 0001
(.venv) PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY>
```

Figure 31: Manual SPN simulation showing substitution and permutation across two rounds.

6.2 Class Activity 2 – S-Box Analysis



```
(.venv) PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY> & "e:\PROJECTS\ADVANCED CRYPTOGRAPHY\.venv\Scripts\python.exe" "e:/PROJECTS/ADVANCED CRYPTOGRAPHY/WEEK 6/sbox_analysis.py"
===== S-BX ANALYSIS =====

S-Box Mappings
0 -> 14
1 -> 4
2 -> 13
3 -> 1
4 -> 2
5 -> 15
6 -> 11
7 -> 8

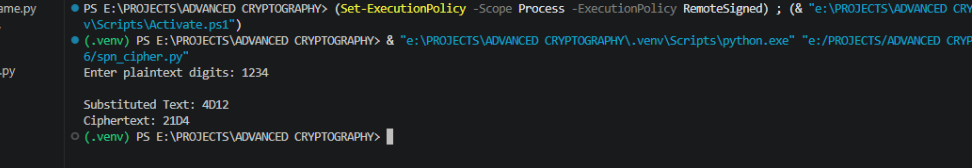
No repeated outputs detected.
S-Box appears secure.
(.venv) PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY>
```

Figure 32: S-Box analysis showing mapping uniqueness and security evaluation

6.3 Feistel vs SPN Comparison

Figure 33: Feistel and SPN structure comparison.

6.4 Implementation of a Simple Substitution-Permutation Network (SPN)



The screenshot shows a Windows terminal window with the following content:

```

vigenere_cipher.py
WEEK 3
  randomness_game.py
  randomness.py
WEEK 4
  aes_encryption.py
  decrypted.txt
  encrypted.bin
  recovered.txt
  secret.txt
  secure_file_locker.py
WEEK 5
  private.pem
  public.pem
  rsa_encryption.py
  secure_messenger.py
WEEK 6
  spn_cipher.py
  .gitignore
  ADVANCED CRYPTOGRAPHY...
  README.md

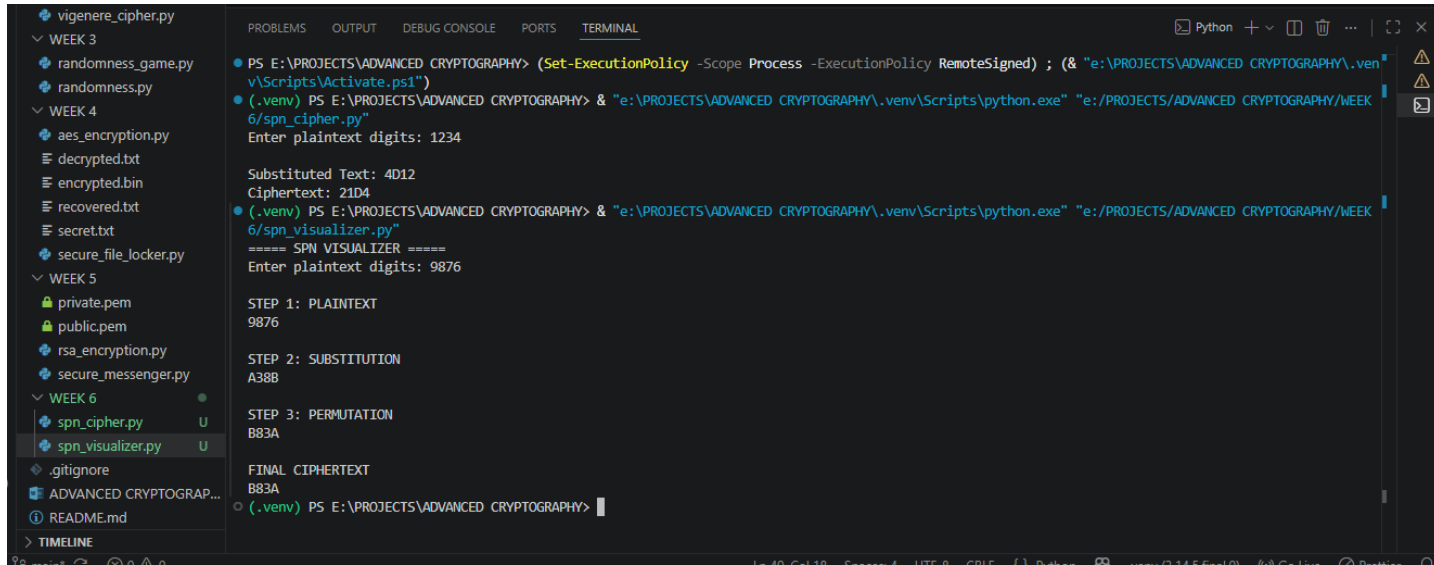
TIMELINE
Python + + + + +
PROBLEMS OUTPUT DEBUG CONSOLE PORTS TERMINAL
PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& "e:\PROJECTS\ADVANCED CRYPTOGRAPHY\.venv\Scripts\Activate.ps1")
(.venv) PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY> & "e:\PROJECTS\ADVANCED CRYPTOGRAPHY\.venv\Scripts\python.exe" "e:\PROJECTS\ADVANCED CRYPTOGRAPHY\WEEK 6/spn_cipher.py"
Enter plaintext digits: 1234

Substituted Text: 4012
Ciphertext: 21D4
(.venv) PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY>

```

Figure 34: Simple SPN cipher implementation and execution.

6.5 Creative Task 1 - SPN Visualizer



```
PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& "e:\PROJECTS\ADVANCED CRYPTOGRAPHY\venv\Scripts\Activate.ps1")
(.venv) PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY> & "e:\PROJECTS\ADVANCED CRYPTOGRAPHY\venv\Scripts\python.exe" "e:/PROJECTS/ADVANCED CRYPTOGRAPHY/WEEK 6/spn_cipher.py"
Enter plaintext digits: 1234

Substituted Text: 4012
Ciphertext: 2104
(.venv) PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY> & "e:\PROJECTS\ADVANCED CRYPTOGRAPHY\venv\Scripts\python.exe" "e:/PROJECTS/ADVANCED CRYPTOGRAPHY/WEEK 6/spn_visualizer.py"
===== SPN VISUALIZER =====
Enter plaintext digits: 9876

STEP 1: PLAINTEXT
9876

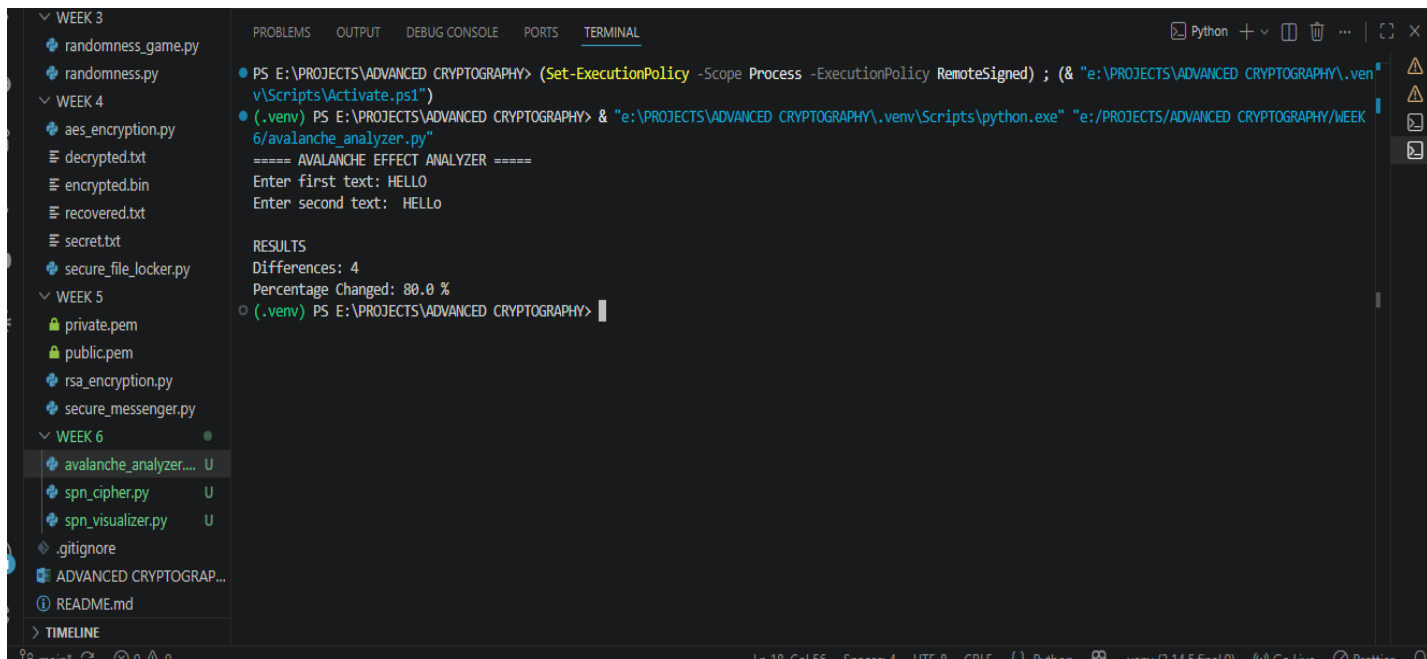
STEP 2: SUBSTITUTION
A38B

STEP 3: PERMUTATION
B83A

FINAL CIPHERTEXT
B83A
(.venv) PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY>
```

Figure 35: SPN Visualizer illustrating the step-by-step transformation of plaintext through substitution and permutation stages to produce ciphertext.

6.6 Creative Task 2 - Avalanche Effect Analyzer



```
PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& "e:\PROJECTS\ADVANCED CRYPTOGRAPHY\venv\Scripts\Activate.ps1")
(.venv) PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY> & "e:\PROJECTS\ADVANCED CRYPTOGRAPHY\venv\Scripts\python.exe" "e:/PROJECTS/ADVANCED CRYPTOGRAPHY/WEEK 6/avalanche_analyzer.py"
===== AVALANCHE EFFECT ANALYZER =====
Enter first text: HELLO
Enter second text: HELLo

RESULTS
Differences: 4
Percentage Changed: 80.0 %
(.venv) PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY>
```

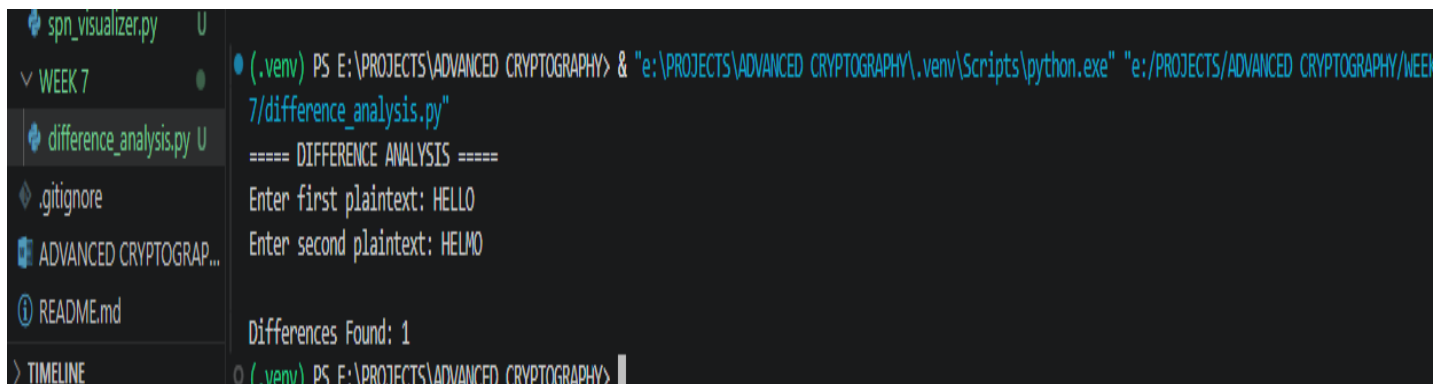
Figure 36: Avalanche Effect Analyzer showing input difference analysis.

6.6.1 Reflection

Week 6 explored Substitution-Permutation Networks through manual simulations, S-Box analysis, and SPN implementation. The creative tasks enhanced understanding of cipher structure and demonstrated how small input changes can influence cryptographic outputs.

7 WEEK 7 - Cryptanalysis Techniques

7.1 Class Activity 1: Difference Analysis



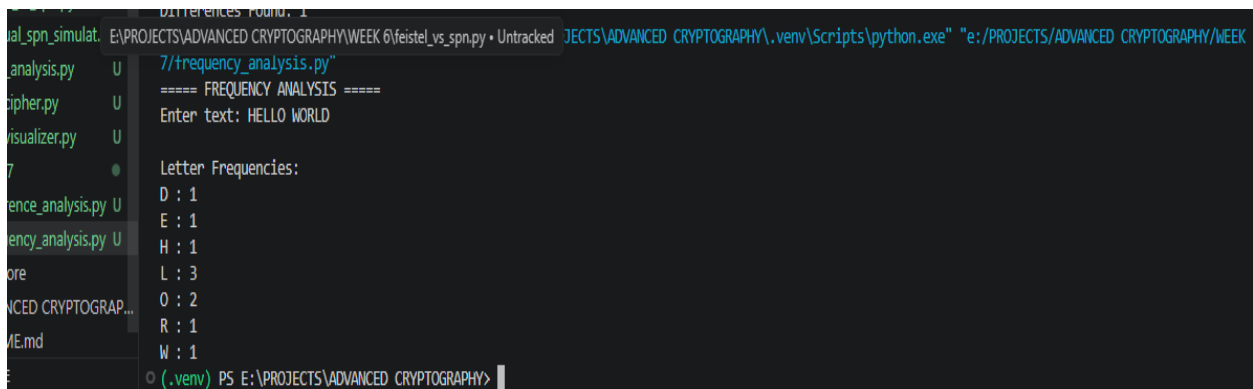
```
spn_visualizer.py U
WEEK 7
difference_analysis.py U
.gitignore
ADVANCED CRYPTOGRAP...
README.md
TIMELINE

(.venv) PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY> & "e:\PROJECTS\ADVANCED CRYPTOGRAPHY\.venv\Scripts\python.exe" "e:/PROJECTS/ADVANCED CRYPTOGRAPHY/WEEK 7/difference_analysis.py"
===== DIFFERENCE ANALYSIS =====
Enter first plaintext: HELLO
Enter second plaintext: HELMO

Differences Found: 1
(.venv) PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY>
```

Figure 37: Difference analysis implementation and execution.

7.2 Class Activity 2: Frequency Analysis



```
... Differences Found: 1
... E:\PROJECTS\ADVANCED CRYPTOGRAPHY\WEEK 6\feistel_vs_spn.py • Untracked JECTS\ADVANCED CRYPTOGRAPHY\.venv\Scripts\python.exe" "e:/PROJECTS/ADVANCED CRYPTOGRAPHY/WEEK 7/frequency_analysis.py"
... analysis.py U
... cipher.py U
... visualizer.py U
... 7
... ence_analysis.py U
... ency_analysis.py U
... ore
... VANCED CRYPTOGRAP...
... ME.md
... E

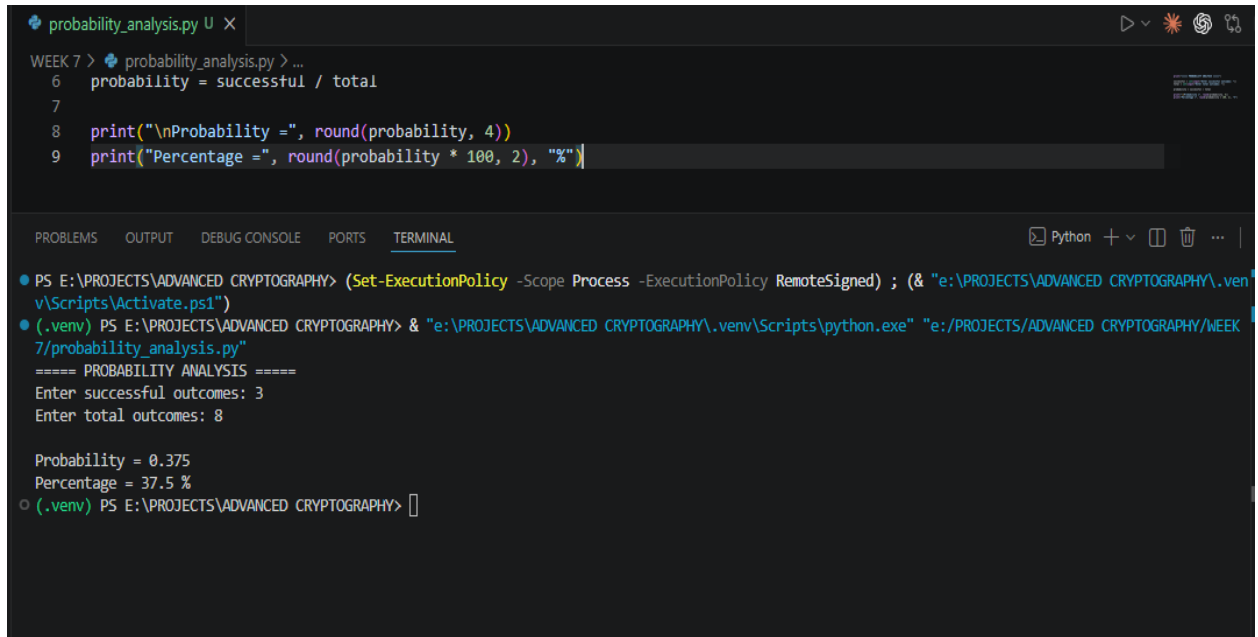
(.venv) PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY>

===== FREQUENCY ANALYSIS =====
Enter text: HELLO WORLD

Letter Frequencies:
D : 1
E : 1
H : 1
L : 3
O : 2
R : 1
W : 1
```

Figure 38 : Frequency analysis showing character occurrence counts.

7.3 Class Activity 3: Probability Analysis



The image shows a Visual Studio Code editor window with a file named `probability_analysis.py`. The script contains the following Python code:

```
WEEK 7 > probability_analysis.py > ...
6 probability = successful / total
7
8 print("\nProbability =", round(probability, 4))
9 print("Percentage =", round(probability * 100, 2), "%")
```

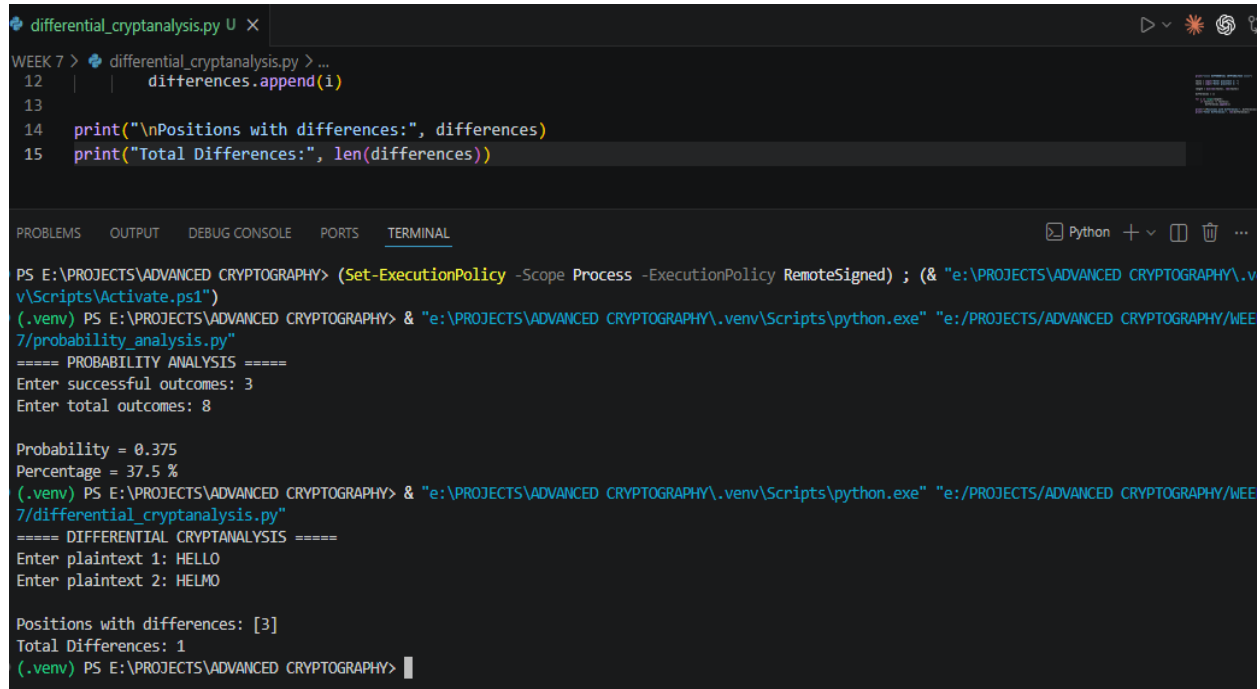
Below the editor, the `TERMINAL` panel is active, showing the command prompt output of the script execution:

```
PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& "e:\PROJECTS\ADVANCED CRYPTOGRAPHY\.venv\Scripts\Activate.ps1")
(.venv) PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY> & "e:\PROJECTS\ADVANCED CRYPTOGRAPHY\.venv\Scripts\python.exe" "e:/PROJECTS/ADVANCED CRYPTOGRAPHY/WEEK 7/probability_analysis.py"
===== PROBABILITY ANALYSIS =====
Enter successful outcomes: 3
Enter total outcomes: 8

Probability = 0.375
Percentage = 37.5 %
(.venv) PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY>
```

Figure 39: Probability analysis showing likelihood calculation.

7.4 Class Activity 4: Differential Cryptanalysis Simulation



```
WEEK 7 > differential_cryptanalysis.py > ...
12 | | differences.append(i)
13
14 print("\nPositions with differences:", differences)
15 print("Total Differences:", len(differences))

PROBLEMS OUTPUT DEBUG CONSOLE PORTS TERMINAL
Python + v [] [] ...

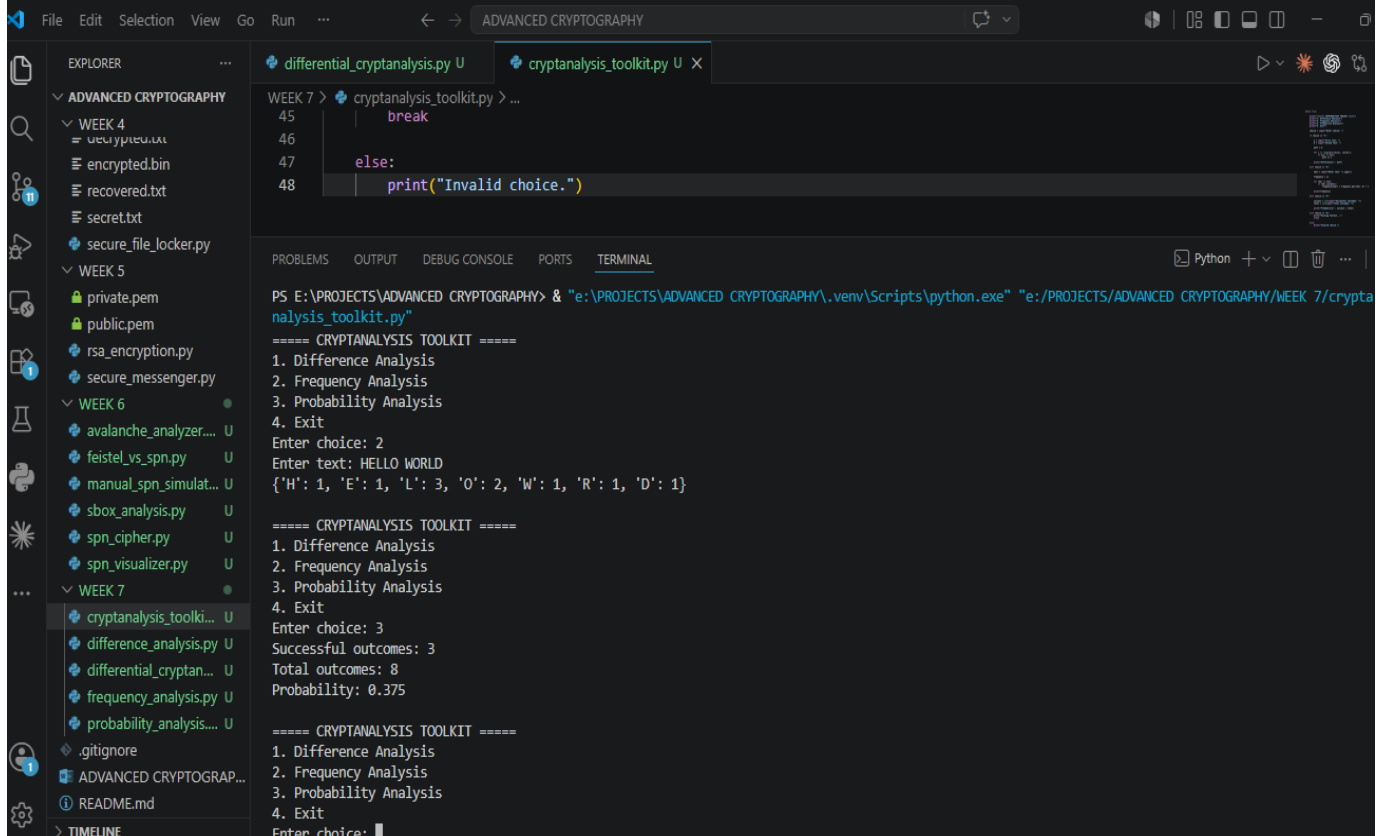
PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& "e:\PROJECTS\ADVANCED CRYPTOGRAPHY\.venv\Scripts\Activate.ps1")
(.venv) PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY> & "e:\PROJECTS\ADVANCED CRYPTOGRAPHY\.venv\Scripts\python.exe" "e:/PROJECTS/ADVANCED CRYPTOGRAPHY/WEEK 7/probability_analysis.py"
===== PROBABILITY ANALYSIS =====
Enter successful outcomes: 3
Enter total outcomes: 8

Probability = 0.375
Percentage = 37.5 %
(.venv) PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY> & "e:\PROJECTS\ADVANCED CRYPTOGRAPHY\.venv\Scripts\python.exe" "e:/PROJECTS/ADVANCED CRYPTOGRAPHY/WEEK 7/differential_cryptanalysis.py"
===== DIFFERENTIAL CRYPTANALYSIS =====
Enter plaintext 1: HELLO
Enter plaintext 2: HELMO

Positions with differences: [3]
Total Differences: 1
(.venv) PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY>
```

Figure 40: Differential cryptanalysis simulation showing plaintext differences.

7.5 Creative Task – Cryptanalysis Toolkit



```
File Edit Selection View Go Run ... ADVANCED CRYPTOGRAPHY

EXPLORER
  ADVANCED CRYPTOGRAPHY
    WEEK 4
      encrypted.bin
      recovered.txt
      secret.txt
      secure_file_locker.py
    WEEK 5
      private.pem
      public.pem
      rsa_encryption.py
      secure_messenger.py
    WEEK 6
      avalanche_analyzer.py
      feistel_vs_spn.py
      manual_spn_simulation.py
      sbox_analysis.py
      spn_cipher.py
      spn_visualizer.py
    WEEK 7
      cryptanalysis_toolkit.py
      difference_analysis.py
      differential_cryptanalysis.py
      frequency_analysis.py
      probability_analysis.py
      .gitignore
      ADVANCED CRYPTOGRAPHY
      README.md
    > TIMELINE

WEEK 7 > cryptanalysis_toolkit.py > ...
45     break
46
47     else:
48         print("Invalid choice.")

PROBLEMS OUTPUT DEBUG CONSOLE PORTS TERMINAL
Python + - - - - -

PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY> & "e:\PROJECTS\ADVANCED CRYPTOGRAPHY\.venv\Scripts\python.exe" "e:/PROJECTS/ADVANCED CRYPTOGRAPHY/WEEK 7/cryptanalysis_toolkit.py"
===== CRYPTANALYSIS TOOLKIT =====
1. Difference Analysis
2. Frequency Analysis
3. Probability Analysis
4. Exit
Enter choice: 2
Enter text: HELLO WORLD
{'H': 1, 'E': 1, 'L': 3, 'O': 2, 'W': 1, 'R': 1, 'D': 1}

===== CRYPTANALYSIS TOOLKIT =====
1. Difference Analysis
2. Frequency Analysis
3. Probability Analysis
4. Exit
Enter choice: 3
Successful outcomes: 3
Total outcomes: 8
Probability: 0.375

===== CRYPTANALYSIS TOOLKIT =====
1. Difference Analysis
2. Frequency Analysis
3. Probability Analysis
4. Exit
Enter choice: 
```

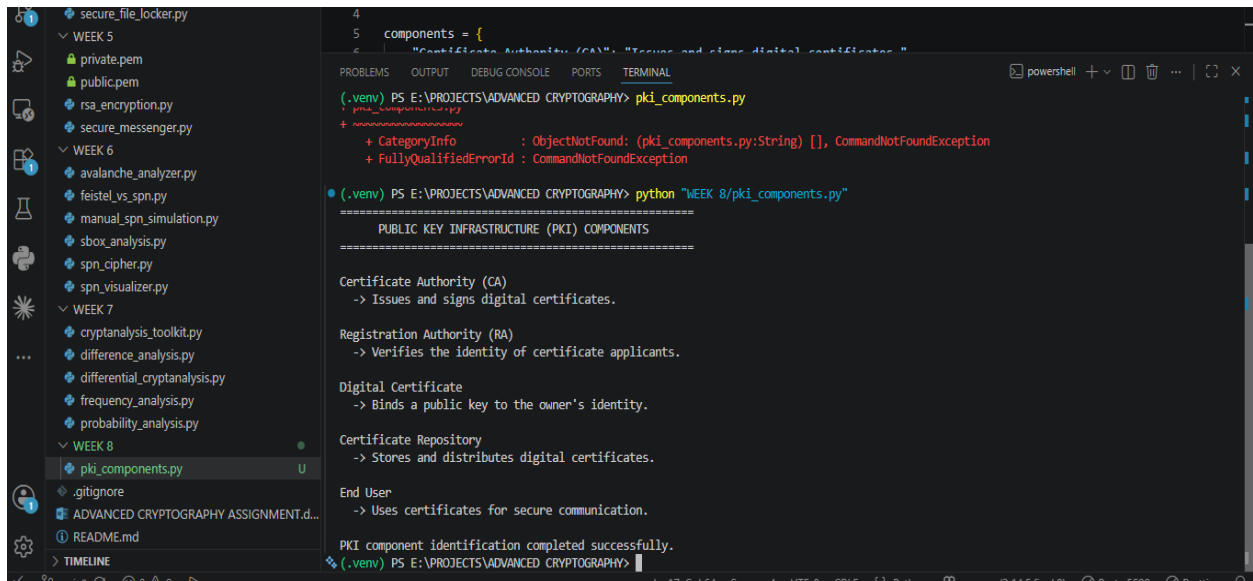
Figure 41: Cryptanalysis Toolkit integrating difference, frequency, and probability analysis techniques.

7.5.1 REFLECTION

Week 7 explored cryptanalysis techniques including difference analysis, frequency analysis, probability calculations, and differential cryptanalysis. The Cryptanalysis Toolkit combined these methods into a single application, improving understanding of how cryptographic systems can be evaluated and attacked.

8 WEEK 8: Public Key Infrastructure (PKI), Digital Certificates and Diffie–Hellman Key Exchange

8.1 Class Activity 1: PKI Component Identification



The screenshot shows a code editor with a file explorer on the left and a terminal window on the right. The file explorer lists files for WEEK 5, WEEK 6, WEEK 7, and WEEK 8. The terminal window displays the output of a Python script named `pk_i_components.py`. The output lists the major components of Public Key Infrastructure (PKI) and their roles in secure digital communication.

```
4 components = {  
5     "Certificate Authority (CA)": "Verifies and signs digital certificates"  
6 }  
7  
8  
9  
10  
11  
12  
13  
14  
15  
16  
17  
18  
19  
20  
21  
22  
23  
24  
25  
26  
27  
28  
29  
30  
31  
32  
33  
34  
35  
36  
37  
38  
39  
40  
41  
42  
43  
44  
45  
46  
47  
48  
49  
50  
51  
52  
53  
54  
55  
56  
57  
58  
59  
60  
61  
62  
63  
64  
65  
66  
67  
68  
69  
70  
71  
72  
73  
74  
75  
76  
77  
78  
79  
80  
81  
82  
83  
84  
85  
86  
87  
88  
89  
90  
91  
92  
93  
94  
95  
96  
97  
98  
99  
100  
101  
102  
103  
104  
105  
106  
107  
108  
109  
110  
111  
112  
113  
114  
115  
116  
117  
118  
119  
120  
121  
122  
123  
124  
125  
126  
127  
128  
129  
130  
131  
132  
133  
134  
135  
136  
137  
138  
139  
140  
141  
142  
143  
144  
145  
146  
147  
148  
149  
150  
151  
152  
153  
154  
155  
156  
157  
158  
159  
160  
161  
162  
163  
164  
165  
166  
167  
168  
169  
170  
171  
172  
173  
174  
175  
176  
177  
178  
179  
180  
181  
182  
183  
184  
185  
186  
187  
188  
189  
190  
191  
192  
193  
194  
195  
196  
197  
198  
199  
200  
201  
202  
203  
204  
205  
206  
207  
208  
209  
210  
211  
212  
213  
214  
215  
216  
217  
218  
219  
220  
221  
222  
223  
224  
225  
226  
227  
228  
229  
230  
231  
232  
233  
234  
235  
236  
237  
238  
239  
240  
241  
242  
243  
244  
245  
246  
247  
248  
249  
250  
251  
252  
253  
254  
255  
256  
257  
258  
259  
260  
261  
262  
263  
264  
265  
266  
267  
268  
269  
270  
271  
272  
273  
274  
275  
276  
277  
278  
279  
280  
281  
282  
283  
284  
285  
286  
287  
288  
289  
290  
291  
292  
293  
294  
295  
296  
297  
298  
299  
300  
301  
302  
303  
304  
305  
306  
307  
308  
309  
310  
311  
312  
313  
314  
315  
316  
317  
318  
319  
320  
321  
322  
323  
324  
325  
326  
327  
328  
329  
330  
331  
332  
333  
334  
335  
336  
337  
338  
339  
340  
341  
342  
343  
344  
345  
346  
347  
348  
349  
350  
351  
352  
353  
354  
355  
356  
357  
358  
359  
360  
361  
362  
363  
364  
365  
366  
367  
368  
369  
370  
371  
372  
373  
374  
375  
376  
377  
378  
379  
380  
381  
382  
383  
384  
385  
386  
387  
388  
389  
390  
391  
392  
393  
394  
395  
396  
397  
398  
399  
400  
401  
402  
403  
404  
405  
406  
407  
408  
409  
410  
411  
412  
413  
414  
415  
416  
417  
418  
419  
420  
421  
422  
423  
424  
425  
426  
427  
428  
429  
430  
431  
432  
433  
434  
435  
436  
437  
438  
439  
440  
441  
442  
443  
444  
445  
446  
447  
448  
449  
450  
451  
452  
453  
454  
455  
456  
457  
458  
459  
460  
461  
462  
463  
464  
465  
466  
467  
468  
469  
470  
471  
472  
473  
474  
475  
476  
477  
478  
479  
480  
481  
482  
483  
484  
485  
486  
487  
488  
489  
490  
491  
492  
493  
494  
495  
496  
497  
498  
499  
500  
501  
502  
503  
504  
505  
506  
507  
508  
509  
510  
511  
512  
513  
514  
515  
516  
517  
518  
519  
520  
521  
522  
523  
524  
525  
526  
527  
528  
529  
530  
531  
532  
533  
534  
535  
536  
537  
538  
539  
540  
541  
542  
543  
544  
545  
546  
547  
548  
549  
550  
551  
552  
553  
554  
555  
556  
557  
558  
559  
560  
561  
562  
563  
564  
565  
566  
567  
568  
569  
570  
571  
572  
573  
574  
575  
576  
577  
578  
579  
580  
581  
582  
583  
584  
585  
586  
587  
588  
589  
590  
591  
592  
593  
594  
595  
596  
597  
598  
599  
600  
601  
602  
603  
604  
605  
606  
607  
608  
609  
610  
611  
612  
613  
614  
615  
616  
617  
618  
619  
620  
621  
622  
623  
624  
625  
626  
627  
628  
629  
630  
631  
632  
633  
634  
635  
636  
637  
638  
639  
640  
641  
642  
643  
644  
645  
646  
647  
648  
649  
650  
651  
652  
653  
654  
655  
656  
657  
658  
659  
660  
661  
662  
663  
664  
665  
666  
667  
668  
669  
670  
671  
672  
673  
674  
675  
676  
677  
678  
679  
680  
681  
682  
683  
684  
685  
686  
687  
688  
689  
690  
691  
692  
693  
694  
695  
696  
697  
698  
699  
700  
701  
702  
703  
704  
705  
706  
707  
708  
709  
710  
711  
712  
713  
714  
715  
716  
717  
718  
719  
720  
721  
722  
723  
724  
725  
726  
727  
728  
729  
730  
731  
732  
733  
734  
735  
736  
737  
738  
739  
740  
741  
742  
743  
744  
745  
746  
747  
748  
749  
750  
751  
752  
753  
754  
755  
756  
757  
758  
759  
760  
761  
762  
763  
764  
765  
766  
767  
768  
769  
770  
771  
772  
773  
774  
775  
776  
777  
778  
779  
780  
781  
782  
783  
784  
785  
786  
787  
788  
789  
790  
791  
792  
793  
794  
795  
796  
797  
798  
799  
800  
801  
802  
803  
804  
805  
806  
807  
808  
809  
810  
811  
812  
813  
814  
815  
816  
817  
818  
819  
820  
821  
822  
823  
824  
825  
826  
827  
828  
829  
830  
831  
832  
833  
834  
835  
836  
837  
838  
839  
840  
841  
842  
843  
844  
845  
846  
847  
848  
849  
850  
851  
852  
853  
854  
855  
856  
857  
858  
859  
860  
861  
862  
863  
864  
865  
866  
867  
868  
869  
870  
871  
872  
873  
874  
875  
876  
877  
878  
879  
880  
881  
882  
883  
884  
885  
886  
887  
888  
889  
890  
891  
892  
893  
894  
895  
896  
897  
898  
899  
900  
901  
902  
903  
904  
905  
906  
907  
908  
909  
910  
911  
912  
913  
914  
915  
916  
917  
918  
919  
920  
921  
922  
923  
924  
925  
926  
927  
928  
929  
930  
931  
932  
933  
934  
935  
936  
937  
938  
939  
940  
941  
942  
943  
944  
945  
946  
947  
948  
949  
950  
951  
952  
953  
954  
955  
956  
957  
958  
959  
960  
961  
962  
963  
964  
965  
966  
967  
968  
969  
970  
971  
972  
973  
974  
975  
976  
977  
978  
979  
980  
981  
982  
983  
984  
985  
986  
987  
988  
989  
990  
991  
992  
993  
994  
995  
996  
997  
998  
999  
1000
```

Output of `pk_i_components.py`:

```
=====
PUBLIC KEY INFRASTRUCTURE (PKI) COMPONENTS
=====

Certificate Authority (CA)
-> Issues and signs digital certificates.

Registration Authority (RA)
-> Verifies the identity of certificate applicants.

Digital Certificate
-> Binds a public key to the owner's identity.

Certificate Repository
-> Stores and distributes digital certificates.

End User
-> Uses certificates for secure communication.

PKI component identification completed successfully.
```

Figure 42: PKI component identification displaying the major components and their roles in secure digital communication.

8.2 Class Activity 2 – Browser Certificate Analysis

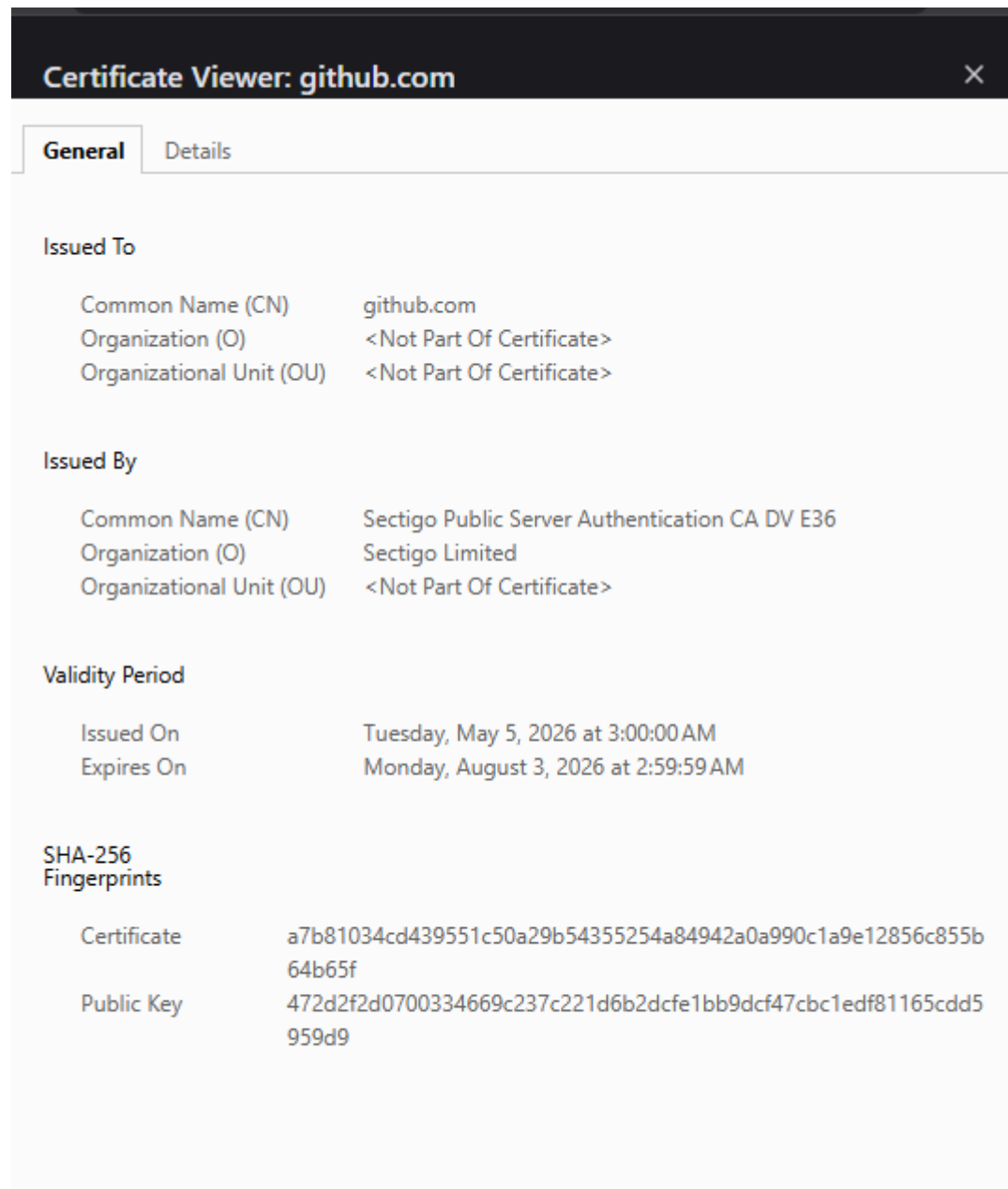


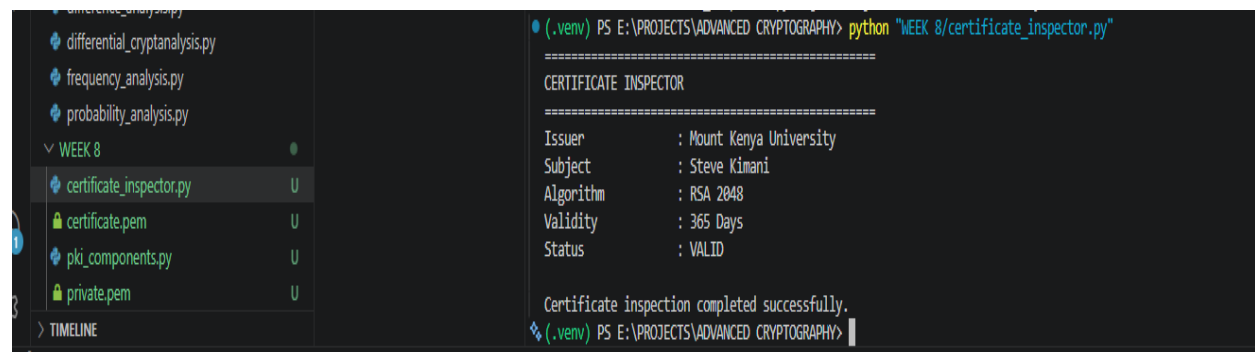
Figure 43: Browser certificate analysis showing the certificate issuer, subject, and validity period.

8.3 OpenSSL Certificate Inspection

```
11th@HIM MINGW64 /e/PROJECTS/ADVANCED CRYPTOGRAPHY/WEEK 8 (main)
openssl x509 -in certificate.pem -text -noout
Certificate:
  Data:
    Version: 3 (0x2)
    Serial Number:
      78:c3:fe:90:ce:83:23:e6:63:04:55:5f:5e:f2:21:ee:b8:05:2a:f4
    Signature Algorithm: sha256WithRSAEncryption
    Issuer: C=KE, ST=Nairobi, L=Nairobi, O=Mount Kenya University, OU=Advanced Cryptography, CN=Geoffrey Mosiria, emailAddress=allthingsjay985@gmail.com
    Validity
      Not Before: Jun 29 04:16:30 2026 GMT
      Not After : Jun 29 04:16:30 2027 GMT
    Subject: C=KE, ST=Nairobi, L=Nairobi, O=Mount Kenya University, OU=Advanced Cryptography, CN=Geoffrey Mosiria, emailAddress=allthingsjay985@gmail.com
    Subject Public Key Info:
      Public Key Algorithm: rsaEncryption
      Public-Key: (2048 bit)
      Modulus:
        00:b6:8e:63:d6:04:0b:ec:db:d2:c2:92:d1:71:ed:
        12:39:86:1f:bc:a5:90:47:d3:8f:01:03:be:de:41:
        98:ba:1e:26:47:bb:34:06:5c:c2:8a:f4:c7:59:1d:
        35:31:a0:0b:da:fd:e3:89:df:23:5c:3c:35:02:d3:
        b1:f0:25:3d:48:e8:06:f4:95:48:b3:72:48:35:6f:
        b2:d4:a0:e2:c2:bf:b5:e5:53:8d:fd:cb:bd:5d:24:
        7e:d1:67:56:5e:16:bd:c8:db:87:c9:c3:37:fe:d6:
        d2:0b:38:10:10:9b:e6:68:c8:20:d2:b0:1c:6d:ee:
        35:57:b0:b2:15:32:1e:8a:9d:f2:77:40:1d:0c:10:
        42:b7:85:96:d4:d3:2e:8f:2d:89:d5:a8:79:a1:09:
        b1:7b:6d:eb:a2:70:7c:05:61:ea:fa:7b:e1:1f:12:
        50:3e:4c:ac:cc:8b:b3:26:7c:a5:9d:36:cc:e2:c2:
        11:7b:98:a9:56:29:58:02:b4:26:9a:18:13:38:83:
        ac:2e:7e:c0:e5:27:51:4b:4b:50:db:aac:f7:e5:bce:
        c7:ef:1a:44:6f:42:b7:04:63:2c:c6:fa:a8:05:56:
        3d:25:35:52:05:29:21:f6:cf:1b:55:e3:bf:a7:if:
        28:a2:1d:b4:9f:4f:66:9c:11:46:cc:e0:0a:04:42:
        bf:4d
      Exponent: 65537 (0x10001)
    X509v3 extensions:
      X509v3 Subject Key Identifier:
        22:5F:80:1F:60:07:39:1B:00:ED:83:C0:F8:2F:88:3F:A1:F2:A9:CB
      X509v3 Authority Key Identifier:
        22:5F:80:1F:60:07:39:1B:00:ED:83:C0:F8:2F:88:3F:A1:F2:A9:CB
      X509v3 Basic Constraints: critical
        CA:TRUE
    Signature Algorithm: sha256WithRSAEncryption
    Signature Value:
      25:4f:73:6b:91:62:18:0d:ac:a1:0c:ad:f0:6f:6a:11:c0:22:
      14:c7:6c:52:ab:81:f3:e8:65:e9:ff:ba:f3:8f:7d:ae:c3:15:
      cd:db:68:b9:52:43:93:5b:22:22:84:cc:83:6d:61:69:ce:e5:
```

Figure 44: OpenSSL certificate inspection displaying certificate details and validity.

8.4 Creative Task 1 – Certificate Inspector



```
(.venv) PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY> python "WEEK 8/certificate_inspector.py"

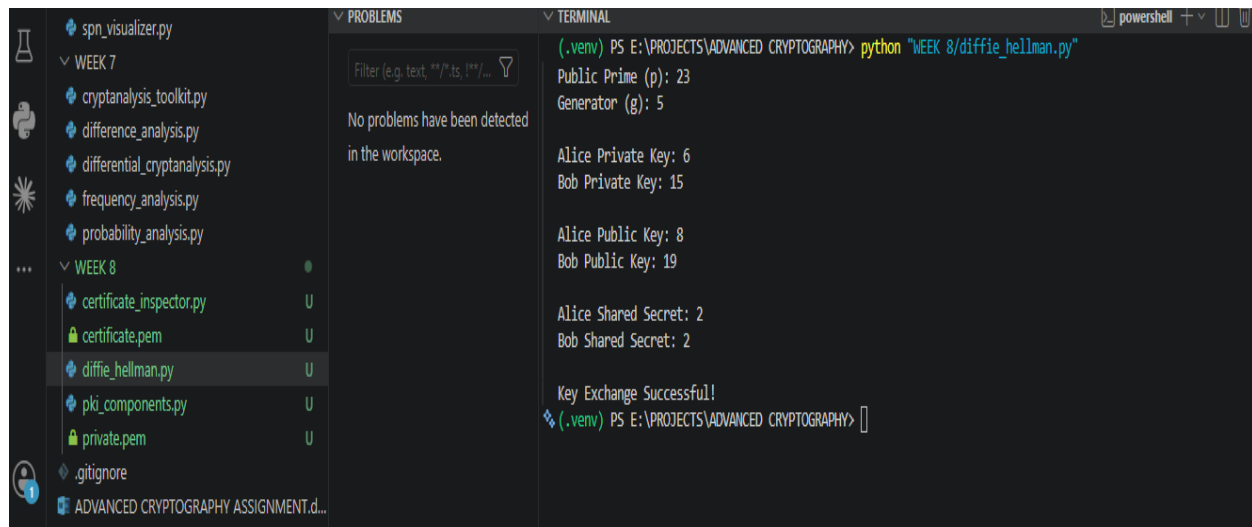
=====
CERTIFICATE INSPECTOR
=====

Issuer       : Mount Kenya University
Subject      : Steve Kimani
Algorithm    : RSA 2048
Validity     : 365 Days
Status       : VALID

Certificate inspection completed successfully.
(.venv) PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY>
```

Figure 45: Certificate Inspector displaying certificate information in a structured format.

8.5 Practical Activity – Diffie–Hellman Key Exchange



The screenshot shows a VS Code interface with a file explorer on the left, a problems panel in the middle, and a terminal on the right. The file explorer shows a project structure with files like `spn_visualizer.py`, `WEEK 7` folder, `cryptanalysis_toolkit.py`, `difference_analysis.py`, `differential_cryptanalysis.py`, `frequency_analysis.py`, `probability_analysis.py`, `WEEK 8` folder, `certificate_inspector.py`, `certificate.pem`, `diffie_hellman.py`, `pkc_components.py`, `private.pem`, `.gitignore`, and `ADVANCED CRYPTOGRAPHY ASSIGNMENT.d...`. The problems panel shows "No problems have been detected in the workspace." The terminal shows the execution of `python "WEEK 8/diffie_hellman.py"` in a PowerShell prompt. The output is as follows:

```
(.venv) PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY> python "WEEK 8/diffie_hellman.py"
Public Prime (p): 23
Generator (g): 5

Alice Private Key: 6
Bob Private Key: 15

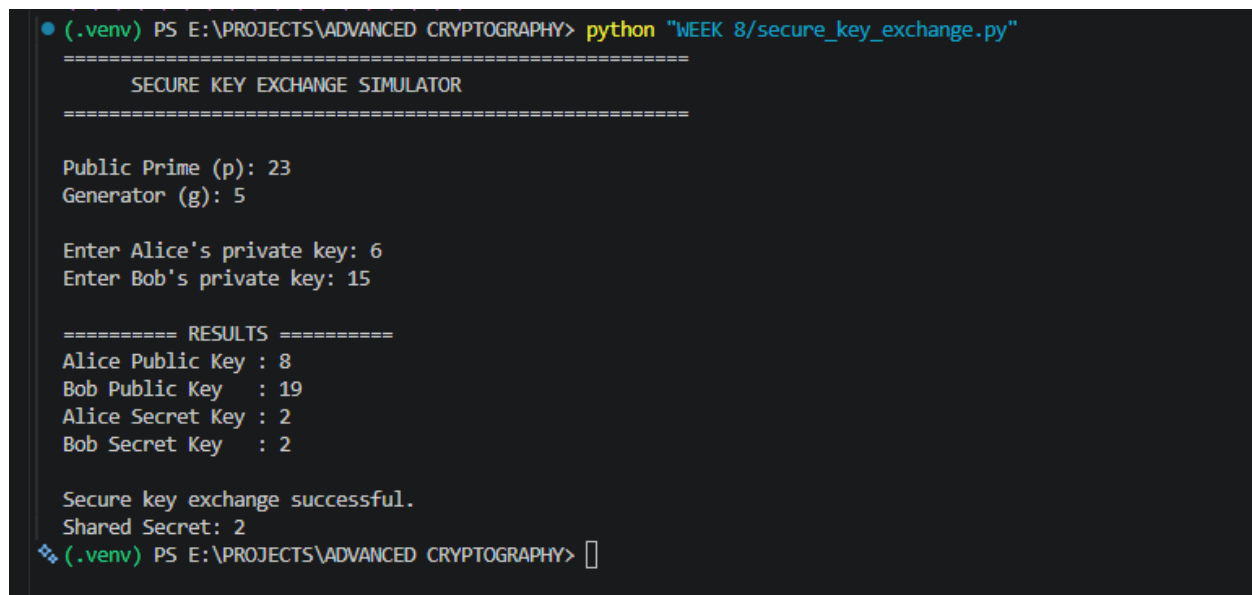
Alice Public Key: 8
Bob Public Key: 19

Alice Shared Secret: 2
Bob Shared Secret: 2

Key Exchange Successful!
(.venv) PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY>
```

Figure 46: Diffie–Hellman key exchange demonstrating successful generation of public keys and a shared secret.

8.6 Creative Task 2 – Secure Key Exchange Simulator



The screenshot shows a VS Code terminal with a PowerShell prompt. The user runs `python "WEEK 8/secure_key_exchange.py"`. The output is as follows:

```
(.venv) PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY> python "WEEK 8/secure_key_exchange.py"
=====
SECURE KEY EXCHANGE SIMULATOR
=====

Public Prime (p): 23
Generator (g): 5

Enter Alice's private key: 6
Enter Bob's private key: 15

===== RESULTS =====
Alice Public Key : 8
Bob Public Key   : 19
Alice Secret Key : 2
Bob Secret Key   : 2

Secure key exchange successful.
Shared Secret: 2
(.venv) PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY>
```

Figure 47: Secure Key Exchange Simulator demonstrating interactive Diffie–Hellman key exchange and shared secret verification.

8.6.1 Reflection

Week 8 strengthened my understanding of PKI, digital certificates, and Diffie–Hellman key exchange. The practical and creative tasks demonstrated how secure communication is established through certificate validation and shared secret generation.